

Analog Devices, Inc.; Antmicro

Zephelin - Zephyr AI Profiling Library - documentation

2026-10-13

CONTENTS

1 Introduction **1**
- 1.1 Documentation outline . . . 1
- 1.2 Zephelin . . . 1
- 1.3 Use cases for Zephelin . . . 2
- 1.4 Zephelin trace collection . . . 2

2 Using and customizing Zephelin **3**
- 2.1 Initializing the workspace . . . 3
- 2.2 Running a sample project with Zephelin . . . 5
- 2.3 Customizing and using Zephelin . . . 6
- 2.4 Testing Zephelin . . . 10

3 Zephelin configuration **11**
- 3.1 Configuring the library . . . 11
- 3.2 Trace formats . . . 14
- 3.3 Trace backends . . . 14
- 3.4 Profiling tiers . . . 14
- 3.5 Adding new configurations . . . 15

4 Memory profiling **17**
- 4.1 Memory types . . . 17
- 4.2 Memory event . . . 17

5 Tracing code scopes **19**
- 5.1 Marking the code . . . 19
- 5.2 Enabling/disabling code scopes at runtime . . . 20

6 Trace capture USB interface requirements **22**
- 6.1 Driver requirements . . . 22
- 6.2 Devicetree requirements . . . 22

7 Trace capture debug interface **23**
- 7.1 Physical hardware . . . 23
- 7.2 Renode . . . 23
- 7.3 Common options . . . 24

8 Instrumentation **25**
- 8.1 Capturing trace . . . 25
- 8.2 Instrumentation with Zephelin's features . . . 26
- 8.3 Additional options . . . 26

9 CTF to TEF conversion **28**
9.1 Usage . . . 28
9.2 Events . . . 29

10 Adding support for new AI inference libraries **38**
10.1 Implementing support . . . 38
10.2 Instrumentation subsystem . . . 42

11 Zephelin Trace Viewer **45**
11.1 Requirements . . . 45
11.2 Building . . . 45
11.3 Usage . . . 45

12 Live tracing of events from the board **50**
12.1 Real-time tracing server structure . . . 50
12.2 Prerequisites . . . 50
12.3 Providing traces to the server . . . 52
12.4 Live-tracing controls . . . 52
12.5 Sample collection of traces . . . 52
12.6 Live tracing with instrumentation . . . 53
12.7 Communication flow . . . 54
12.8 Server API reference . . . 54

13 Zephelin usage examples **57**
13.1 Using code scopes . . . 57
13.2 Simple TFLite Micro profiling . . . 58
13.3 Simple microTVM profiling . . . 58
13.4 Full TFLite Micro traces with instrumentation using separate UART connection for instrumentation traces . . . 59
13.5 Full TFLite Micro traces with instrumentation sent through tracing subsystem . . . 60
13.6 Multithreaded application running multiple models . . . 61
13.7 Simple tracing of two applications running on different SoCs with common external clock . . . 62
13.8 Preprocessing to micro-speech pipeline . . . 63

14 Troubleshooting **65**
14.1 Trace capture . . . 65
14.2 Converting traces . . . 66
14.3 Live tracing . . . 66

15 Zephelin's developer guidelines **68**
15.1 General Zephyr development guidelines . . . 68
15.2 Adding build-time and runtime configurations for Zephelin . . . 68
15.3 Adding support for new AI inference libraries . . . 68
15.4 Adding support for new trace backends . . . 68
15.5 Adding support for tracing new data sources in Zephelin . . . 69
15.6 Adding Zephyr patches . . . 72
15.7 Supporting a new Zephyr version . . . 72

16 Zephelin benchmarking **73**
16.1 Zephelin Benchmark Results . . . 73
16.2 Benchmarks . . . 74
16.3 Configs . . . 74

16.4 Commits 76

Index **77**

CHAPTER

ONE

INTRODUCTION

1.1 Documentation outline

This documentation describes the following aspects of Zephelin and the associated projects:

- *Introduction* - provides general information on Zephelin, its use cases and capabilities
- *Using and customizing Zephelin* - describes how to build, test and use the profiling middleware
- *Zephelin configuration* - describes build-time and runtime configuration of Zephelin library
- *Memory profiling* - describes memory profiling, along with memory events
- *Tracing code scopes* - describes the use of tracing code scopes
- *Trace capture debug interface* - provides information on the debug interface for capturing traces
- *Instrumentation* - shows how to use instrumentation subsystem with Zephelin
- *CTF to TEF conversion* - describes how traces are converted and processed
- *Adding support for new AI inference libraries* - depicts how to track a new runtime
- *Zephelin Trace Viewer* - describes the tool for trace visualization and provides sample applications for various tracing features
- *Live tracing of events from the board* - describes how to trace the execution in real-time
- *Zephelin usage examples* - aggregates and describes various example applications present in the project
- *Troubleshooting* - collects the most common problems with building, capturing and converting traces, along with their solutions
- *Zephelin's developer guidelines* - summarizes and links resources regarding development of Zephelin
- *Zephelin benchmarking* - provides details on the overhead introduced by the library based on different options

1.2 Zephelin

Zephyr Profiling Library (ZPL), or Zephelin for short, is a library which enables capturing and reporting runtime performance metrics, for the profiling and detailed analysis of Zephyr applications, with a special focus on applications running AI/ML inference workloads.

It collects traces from execution with either:

- **Tracing subsystem and predefined or user-provided events** - allowing to track functions, sections of code, loops, etc.
- **Instrumentation subsystem** - allowing to leverage the instrumentation feature of a compiler to automatically trace functions defined in the source, with a possibility to filter functions of interest

Zephelin is an official external module for Zephyr RTOS.

1.3 Use cases for Zephelin

Zephelin can be used to analyze:

- Regular Zephyr applications
- Internals of the Zephyr RTOS with the help of the *instrumentation subsystem*
- AI models
- AI runtimes
- Multithreaded applications

1.4 Zephelin trace collection

Zephelin collects:

- Traces
- User-defined code scopes
- CPU load
- Memory usage (from runtime and RAM/ROM reports)
- Die temperature
- Model-related data:
  - Global - inference time
  - Per-layer:
    - Memory consumption
    - Total runtime of each layer, and of each layer type
    - Dimensions of tensors

The above data is later processed by `west zpl-<backend>-capture` commands (described in *Trace collection*) and enhanced using `west zpl-prepare-trace` (described in *CTF and TEF trace processing*).

CHAPTER

TWO

USING AND CUSTOMIZING ZEPHELIN

2.1 Initializing the workspace

First, make sure all dependencies required by Zephyr RTOS are installed - follow the Getting started guide.

Secondly, create a workspace and clone the Zephelin repository:

```
mkdir workspace && cd workspace
git clone --recursive git@github.com:antmicro/zephelin.git
cd zephelin
```

Then, install `west` and additional dependencies listed in the project's `requirements.txt` with `pip`:

```
pip install -r requirements.txt
```

Next, pick the Zephyr version to build against and initialize the workspace using West. Pass the version to the script, or set `ZEPHYR_VERSION` to choose it (the supported values are the file names in `zephyr-versions/`).

```
source ./scripts/zephyr_version.sh v4.4.1
west init -l . --mf "$ZPL_WEST_MANIFEST"
```

Download, patch and prepare the project sources using the following commands:

```
west update
west packages pip --install
west patch -b "$ZPL_PATCH_BASE" -l "$ZPL_PATCH_YML" apply
west zephyr-export
```

Note

For default Zephyr release (`v4.3.0`) can be configured and used using regular calls:

```
west init -l .
west update
west packages pip --install
west patch apply
west zephyr-export
```

The workspace records which version it was initialized for, so from then on `scripts/zephyr_version.sh` picks that version up on its own. To move an existing workspace to another version, run:

```
./scripts/switch_zephyr_version.sh <version>
```

It removes the outgoing version's patches, repoints the manifest, updates the modules and applies the incoming version's patches, in that order.

> **Note**
>
> Two versions may expect different Zephyr SDKs.

For testing without hardware in the loop, download Renode portable and add the download path to the `PATH` environment variable:

> **Warning**
>
> Make sure to use Renode version v1.16.0.4276 or newer.

Linux

Mac OS

```
wget https://builds.renode.io/renode-latest.linux-portable-dotnet.tar.gz
mkdir renode-portable
tar --strip-components=1 -C ./renode-portable -xvf renode-latest.linux-portable-
↪dotnet.tar.gz
export PATH=$(pwd)/renode-portable:$PATH
export PYRENODE_BIN="$PWD/renode-portable/renode"
export PYRENODE_RUNTIME=coreclr
```

```
wget https://dot.net/v1/dotnet-install.sh
chmod +x dotnet-install.sh
./dotnet-install.sh --version 8.0.410
export PATH="$HOME/.dotnet:$PATH"

git clone https://github.com/renode/renode.git
pushd renode
./build.sh --net -t -n --host-arch arm64
popd

export PATH="$(pwd)/renode:$PATH"
export PYRENODE_BUILD_DIR="$PWD/renode"
export PYRENODE_RUNTIME=coreclr
```

Finally, download Zephyr SDK:

```
west sdk install
```

2.2 Running a sample project with Zephelin

To collect traces and visualize them using Zephelin Trace Viewer, you can run a simple demo with gesture recognition, based on the data from an accelerometer. The default configuration in this demo collects traces along with all possible additional information, like memory usage, die temperature, inference statistics, and more. One UART provides logs from the application, whereas the other UART returns CTF traces.

2.2.1 Running the demo in a Renode simulation

To build the demo, run:

```
west build -p -b stm32f746g_disco/stm32f746xx samples/demo
```

To run it in a Renode simulation, run:

```
python ./scripts/run_renode.py \
    --repl ./samples/demo/boards/stm32f746g_disco_lis2ds12.repl \
    --sensor i2c1.lis2ds12 \
    --sensor-samples ./samples/common/data/magic_wand/magic_wand.data \
    --trace-output trace.ctf \
    --timeout 10
```

This demo will run for 10 seconds, until a timeout is reached. Afterwards, CTF traces returned over the secondary UART will be stored in `trace.ctf`.

> **Note**
>
> For trace collection on actual hardware, refer to *Trace collection*.

The trace needs to be converted to a TEF file to be loaded into Zephelin Trace Viewer.

To do that, run:

```
west zpl-prepare-trace ./trace.ctf --tvm-model-path samples/common/tvm/model/
↪magic-wand-graph.json -o ./tef_tvm_profiler.json
```

The `--tvm-model-path` element is an input argument with the path to a TVM model graph, which is used to introduce additional model data to the TEF trace file metadata.

To get an overview of the traces, load the output `tef_tvm_profiler.json` file in Zephelin Trace Viewer.

2.2.2 Running the demo on HW

This demo can be also run on a physical MAX32690 Evaluation Kit using the ADXL345 accelerometer. The accelerometer can be connected to `i2c0` as follows:

- `VIN` -> any 3v3 pin
- `GND` -> any ground pin
- `SDA` -> pin 7 on port `JH4`
- `SCL` -> pin 8 on port `JH4`

It is also required to connect the following jumpers to enable `i2c0`:

- `JP2`
- `JP3`
- `JP4`

To program the board, connect the MAX32625PICO using a USB cable to the PC, and via the `SWD` header to the board. Then, connect another USB cable to `CN2` - this will be used to collect data via UART (`uart2`). Finally, to collect the traces, connect USB-UART converter to `uart0` using following pins:

- `RX` - pin 11 on port `JH4`
- `TX` - pin 12 on port `JH4`

Alternatively, when the USB-UART converter is not available, it is possible to switch UARTs in the board overlay and collect traces the same way as logs - using `uart2`.

To build the demo, run:

```
west build -p -b max32690evkit/max32690/m4 samples/demo
```

And then, to flash the board, run:

```
west flash --openocd=${MSDK_OPENOCD}
```

using `openocd` from Analog Devices MSDK.

After flashing, there should be logged readings from the sensor to the UART.

```
*** Booting Zephyr OS build v4.2.0-rc2-49-g732a3a5c6655 ***
adxl345@53: x=+0.312 y=+0.906 z=+0.046
adxl345@53: x=+0.296 y=+0.906 z=+0.031
adxl345@53: x=+0.312 y=+0.890 z=+0.015
adxl345@53: x=+0.312 y=+0.890 z=+0.015
adxl345@53: x=+0.312 y=+0.890 z=+0.031
...
```

To trigger gesture recognition, move the sensor.

```
...
adxl345@53: x=-0.015 y=+0.374 z=+0.078
model output: wing=1.000 ring=0.000 slope=0.000 negative=0.000
```

Then, the traces collected via UART can be analyzed the same way as in *Running the demo in a Renode simulation*.

2.3 Customizing and using Zephelin

Zephelin can be enabled by y-selecting the `CONFIG_ZPL` symbol in the project configuration file. By default, Zephelin is automatically initialized during Zephyr initialization, but it can also be initialized in a runtime by setting the Kconfig symbol `CONFIG_ZPL_AUTORUN_INIT` to `n` and using the `zpl_init()` function defined by the `zpl/lib.h` header. You can enable various Zephelin components by using Kconfig and runtime configuration, as described in the following sections.

Warning

Manual and automatic initializations are mutually exclusive. Enabling `CONFIG_ZPL_AUTORUN_INIT` makes `zpl_init` local as a protection against using both methods at the same time. Improper use will result in a compile-time error.

2.3.1 Configuration

The library can be configured both during building and while running on a device. To find out how to configure the library and how to add new configurations, check *Zephelin configuration*.

2.3.2 Trace collection

To enable Zephelin tracing support, the user should enable the symbol `CONFIG_ZPL_TRACE` in Kconfig. You can then select one of the following formats:

- Plaintext format, by y-selecting `CONFIG_ZPL_TRACE_FORMAT_PLAINTEXT`
- Common Trace Format (CTF), by y-selecting `CONFIG_ZPL_TRACE_FORMAT_CTF`

You can choose how the traces will be delivered to the host PC by selecting one of the available tracing backends:

- UART, by y-selecting `CONFIG_ZPL_TRACE_BACKEND_UART`
- USB, by y-selecting `CONFIG_ZPL_TRACE_BACKEND_USB`
- Debugger, by y-selecting `CONFIG_ZPL_TRACE_BACKEND_DEBUGGER`
- Renode's simulated trivial UART, by y-selecting `CONFIG_ZPL_TRACE_BACKEND_TRIVIAL_UART`

Depending on the tracing backend used, the following commands can be used for trace capture.

UART

- Config option - `CONFIG_ZPL_TRACE_BACKEND_UART`
- Command:

```
usage: west zpl-uart-capture [-h] [--send-enable] serial_port serial_
↪baudrate output_path

Capture traces using UART. This command captures traces using the serial␣
↪interface.

positional arguments:
  serial_port      Seral port
  serial_baudrate  Seral baudrate
  output_path      Capture output path

options:
  -h, --help       show this help message and exit
  --send-enable    Send 'enable' to device before collecting data to enable␣
↪tracing, requires
                   CONFIG_TRACING_HANDLE_HOST_CMD to be enabled in the app
```

(continues on next page)

(continued from previous page)

```
  --send-to-remote
                        Forward collected data to specified address
```

USB

- Config option - `CONFIG_ZPL_TRACE_BACKEND_USB`
- Command:

```
usage: west zpl-usb-capture [-h] [-t TIMEOUT] [-w] vendor_id product_id␣
↪output_path

Capture traces using USB. This command captures traces using USB.

positional arguments:
  vendor_id             Vendor ID
  product_id            Product ID
  output_path           Capture output path

options:
  -h, --help            show this help message and exit
  -t TIMEOUT, --timeout TIMEOUT
                        Timeout of the USB capture in seconds
  -w, --wait-for-device
                        When this flag is set, the command will wait for the␣
↪device to connect
  --send-to-remote      Forward collected data to specified address
```

Debugger

- Config option - `CONFIG_ZPL_TRACE_BACKEND_DEBUGGER`
- Command:

```
usage: west zpl-gdb-capture [-h] [--elf-path ELF_PATH] [--gdb-port GDB_PORT]␣
↪[--gdb GDB]
                            [--no-debug-server] [--openocd OPENOCD]
                            [--buffer-full | --n-bytes N_BYTES]
                            output_path

Capture traces using GDB. This command captures traces using GDB from RAM␣
↪using the `dump` command.

positional arguments:
  output_path           Capture output path

options:
  -h, --help            show this help message and exit
  --elf-path ELF_PATH   Zephyr ELF path, by default deduced from Zephyr build␣
↪dir
```

(continues on next page)

(continued from previous page)

```
  --gdb-port GDB_PORT  GDB server port
  --gdb GDB            Path to GDB
  --no-debug-server    Don't set up the debug server
  --openocd OPENOCD    Path to custom OpenOCD
  --buffer-full        Run application until trace buffer is full
  --n-bytes N_BYTES    Run application until there is at least n in trace␣
↪buffer
  --send-to-remote     Forward collected data to specified address
```

Trivial UART in Renode

On top of the above, Renode's simulated trivial UART can be used as well to collect traces in a simulation: `CONFIG_ZPL_TRACE_BACKEND_TRIVIAL_UART`.

2.3.3 Adding named events to traces

Zephelin provides methods for introducing custom named events to traces from the source code level. To use named events, include the header `zpl/lib.h`, and use the function `sys_trace_named_event()` to generate named events.

2.3.4 Changing timestamp clock

By default, Zephelin uses kernel cycles for event timestamps. This can be overridden by y-selecting `CONFIG_ZPL_CONFIGURABLE_TIMESTAMP_CLOCK` and passing a structure to `zpl_clock_set`, which has the following definition:

```
// zpl/time.h

typedef struct {
    uint64_t (*cycles_get)(void);
    uint64_t (*timestamp_get)(uint64_t cycles);
} zpl_clock_t;
```

This might be useful when the system clock is not synchronized between different Zephelin instances running at the same time.

An example can be found in samples/basic/custom_clock/src/main.c.

2.3.5 Memory profiler

To use Zephelin's memory profiler, y-select the `CONFIG_ZPL_MEMORY_PROFILING` in Kconfig. No further actions are needed in the application code to generate memory profiling events in the generated trace. Memory profiling, along with memory events, are described in *Memory profiling*.

2.3.6 TLFM events

To use Zephelin's custom events with Tensorflow Lite Micro (TLFM), use the functions `zpl_emit_tflm_begin_event()` and `zpl_emit_tflm_end_event()`, provided by `zpl/tflm_events.h`.

2.3.7 Delayed emission of layer profiling events

By default, the TFLM and microTVM profilers emit a trace event on every operator boundary right when that boundary is reached. Formatting and pushing an event into the tracing buffer takes time, and since it happens between the operators, that time is included in the inference being measured.

The delayed emission mechanism removes this overhead from the measurement. When it is enabled, the profiler only reads the cycle counter at each operator boundary and stores the timestamps in memory. All the events are formatted and emitted at once when the inference finishes.

The mechanism is disabled by default and can be enabled per runtime:

- `CONFIG_ZPL_TFLM_PROFILER_DELAYED_EMISSION` for TFLM,
- `CONFIG_ZPL_TVM_PROFILER_DELAYED_EMISSION` for microTVM.

The number of operator events buffered during a single inference is limited by `CONFIG_ZPL_TFLM_PROFILER_MAX_EVENTS` / `CONFIG_ZPL_TVM_PROFILER_MAX_EVENTS` (32 by default). Operators beyond that limit are not traced, and a warning is logged.

> **Warning**
>
> Delayed emission makes the trace stream non-chronological - the buffered operator events reach the host after events that were emitted while the inference was running (for example code scopes marked from an interrupt), but they carry earlier timestamps.
>
> Offline processing with *CTF to TEF conversion* handles this, because the whole trace is sorted before conversion. *Live tracing of events from the board* does not - it converts and renders the events as they arrive, and the trace viewer drops events whose timestamp is older than the newest one already displayed for a given thread. With delayed emission enabled, the layer profiling events are therefore likely to be lost in live tracing.

2.4 Testing Zephelin

To run unit and integration tests, use the following commands:

```
west twister -v -p max78002evkit/max78002/m4 -p max32690fthr/max32690/m4 -p qemu_
↪cortex_m3 -T samples -T tests
```

CHAPTER

THREE

ZEPHELIN CONFIGURATION

The library can be configured at build-time as well as at runtime.

3.1 Configuring the library

To enable Zephelin's tracing support, enable the symbol `CONFIG_ZPL_TRACE` in Kconfig.

3.1.1 Build-time configuration

The build-time configuration can only be selected during build-time. The configuration can either be appended to the conf files:

```
CONFIG_ZPL_MEMORY_PROFILING=y
```

Or appended to the build command:

```
west build -b stm32f429i_disc1/stm32f429xx samples/trace/tflm_profiler -- -
↪DCONFIG_ZPL_MEMORY_PROFILING=y
```

3.1.2 Runtime configuration

A set of configuration flags defines runtime configuration. Each flag denotes whether a corresponding configuration (e.g. memory profiling) is enabled or disabled.

By default, all flags are set.

Guards

In the code, there are two ways to use the runtime configuration as guards for the functionalities.

Wait-for function

The "wait-for" function stops the thread and waits for a signal. Once the signal arrives, the thread is woken up and continues execution. This is useful when the functionality which this configuration guards is in a separate thread. This "wait-for" functionality can be called using the macro `ZPL_CONF_WAIT`. Example:

```
while (true) {
    ZPL_CONF_WAIT(memory_profiler);
    // ... Guarded code
}
```

> **Note**
>
> The code is guarded only if the given configuration is defined with `threading` parameter set to 1. See *adding new configuration entry* for details.

Is-enabled function

The "is-enabled" functionality doesn't stop the thread, and only checks if the configuration is turned on. The function returns a boolean value, which can be checked in a standard "if", to create a guard for the functionality. The "is-enabled" function can be called using the macro `ZPL_CHECK_IF_CONF`. Example:

```
if (ZPL_CONF_IS_ENABLED(memory_profiler)) {
    // ... Guarded code
}
```

Return-if-disabled function

The "return-if-disabled" is a convenience macro using `ZPL_CONF_IS_ENABLED` to return if it evaluates to `false`. The check-if function can be called using the macro `ZPL_CONF_RETURN_IF_DISABLED`. Example:

```
void zpl_profile_stack(const struct k_thread *thread, void *user_data) {
    ZPL_CONF_RETURN_IF_DISABLED(memory_profiler);
    // ... Guarded code
}
```

Interfaces

The runtime configuration supports the following interfaces:

- C API,
- UART shell commands,
- debug flags.

C API

To toggle configuration from the code, use the `ZPL_CONF_SET` macro, e.g.:

```
#include <zpl/configuration.h>

int main(void) {
    ZPL_CONF_SET(inference, false);
    /* ... */
    ZPL_CONF_SET(inference, true);
    /* ... */
}
```

UART commands

UART commands runtime configuration can be enabled by selecting the Kconfig option `CONFIG_ZPL_RUNTIME_CONFIG_UART`. This option enables runtime configuration via shell module with custom configuration commands.

To display the available configs type `help`:

```
uart:~$ help
Available commands:
  help
  inference
  memory
  memory_profiler
```

Each config can then be either enabled or disabled using the following syntax:

```
<command> enable
<command> disable
```

For example, this would be the memory profiler configuration:

```
memory_profiler enable
memory_profiler disable
```

Debug interface

You can use the debug configuration either directly from GDB, or using the west command `zpl-debug-config`. To use it directly in GDB, make sure to load the ELF file with debug symbols. Then set the desired config to `0` (disable) or `1` (enable):

```
set var debug_configs.<config> = <value>
```

For example, to enable the memory usage tracing, use the command:

```
set var debug_configs.memory_profiler = 1
```

You can also use the west command `zpl-debug-config`:

```
usage: west zpl-debug-config [-h] elf_path config value

Enable/Disable configs in runtime using debug interface. This command can list␣
↪available configs and enable/disable them.

positional arguments:
  elf_path    Zephyr ELF path
  config      Config to set
  value       Value of the config (enable/disable)

options:
  -h, --help  show this help message and exit
```

For example, to enable the memory usage tracing, use the command:

```
west zpl-debug-config build/zephyr/zephyr.elf memory_profiler enable
```

3.2 Trace formats

The user can then select the following formats:

- Plaintext format, by y-selecting `CONFIG_ZPL_TRACE_FORMAT_PLAINTEXT`
- Common Trace Format (CTF), by y-selecting `CONFIG_ZPL_TRACE_FORMAT_CTF`

3.3 Trace backends

You can choose how the traces will be delivered to the host PC by selecting one of the available tracing backends:

- UART, by y-selecting `CONFIG_ZPL_TRACE_BACKEND_UART`
- USB, by y-selecting `CONFIG_ZPL_TRACE_BACKEND_USB`
- Debugger, by y-selecting `CONFIG_ZPL_TRACE_BACKEND_DEBUGGER`
- Renode's simulated trivial UART, by y-selecting `CONFIG_ZPL_TRACE_BACKEND_TRIVIAL_UART`

3.4 Profiling tiers

The library supports three distinct profiling tiers, each offering a different balance of performance impact and tracing detail:

- `ZPL_TRACE_MINIMAL_MODE` - this mode provides extremely lightweight profiling with minimal overhead.
  - It is designed for environments where performance is critical, and persistent trace data is not required.
  - When enabled, it activates basic inference profiling (`ZPL_INFERENCE_PROFILING`) and memory usage tracing (`ZPL_MEMORY_USAGE_TRACE`) to give a high-level view of system behavior without introducing measurable latency.
- `ZPL_TRACE_LAYER_PROFILING_MODE` - this mode enables layer-level timing.
  - It offers more granularity by tracking timing at individual model layers.
- `ZPL_TRACE_FULL_MODE` - this comprehensive profiling mode enables complete tracing.
  - This mode is suitable for in-depth debugging and performance analysis but incurs a higher runtime cost due to its extensive trace capture.
  - It includes all basic and layer-level profiling features, and additionally employs a wide array of kernel and runtime tracing subsystems, which include:
    - syscall tracing,
    - thread scheduling,
    - interrupt service routines (ISRs),
    - synchronization primitives (semaphores, mutexes, condition variables),

* IPC mechanisms (queues, FIFOs, LIFOs, stacks, message queues, mailboxes, pipes),
* memory allocators (heap, memory slabs),
* timers,
* event handling,
* polling,
* power management,
* networking (core, sockets),
* various hardware interfaces (GPIO, idle state tracking).

3.5 Adding new configurations

Depending on the selected Zephelin integration, the source files in projects need to be adjusted, as described in the following subsections.

3.5.1 Build-time configuration

The build-time configurations can be added to the zpl/Kconfig file. The config should follow the Kconfig standard. For example:

```
config ZPL_RUNTIME_CONFIG_UART
    bool "Runtime configuration via UART"
```

It can later be used to decide whether a corresponding functionality should be defined:

- in code, e.g. for function declarations:

```
#ifdef CONFIG_ZPL_RUNTIME_CONFIG_UART
/* ... */
#endif
```

- in CMake, e.g. for including sources with function definitions:

```
zephyr_library_sources_ifdef(CONFIG_ZPL_RUNTIME_CONFIG_UART configuration_uart.c)
```

3.5.2 Runtime configuration

New runtime configurations can be defined in the configuration source files.

New configuration entry

To add a new configuration entry, add it to the list of existing ones in include/zpl/configuration.h, e.g.:

```
#define CONFIGS(CONFIG)                                                 ␣
↪\
    /* ... */                                                           ␣
↪\
    ZPL_IMPL_IF_ENABLED(CONFIG_ZPL_MEMORY_PROFILING,    CONFIG)         ␣
```

(continues on next page)

(continued from previous page)

```
↪\
        (memory_profiler, IS_ENABLED(CONFIG_ZPL_MEMORY_PROFILING_CONF_THREADING))␣
↪\
    /* ... */
```

where:

- `CONFIG` is the macro defining configuration functionality,
- `ZPL_IMPL_IF_ENABLED(CONFIG_ZPL_MEMORY_PROFILING, CONFIG)` decides whether a provided `CONFIG` should be expanded,
- `(memory_profiler, IS_ENABLED(CONFIG_ZPL_MEMORY_PROFILING_CONF_THREADING))` defines `CONFIG` arguments:
  1. name of the configuration,
  2. threading flag indicating whether a given configuration should define mutexes, conditional variables, and thread-safe mechanisms.

This will automatically attach existing logic to a given configuration entry.

New configuration functionality

To extend the configuration capabilities, add new `CONFIGS` invocations or change the definition of an argument macro:

- adding declaration:

```
// include/zpl/configuration.h

#define ZPL_CONF_WAIT_DECLARE(name)  /* ... */
#define ZPL_CONF_DECLARE(name, ...) \
    /* ... */                       \
    ZPL_CONF_WAIT_DECLARE(name)     \
    /* ... */
CONFIGS(ZPL_CONF_DECLARE)
```

- adding definition:

```
// zpl/configuration.c

#define ZPL_CONF_WAIT_DEFINE(name)  /* ... */
#define ZPL_CONF_DEFINE(name, threading, ...) \
    /* ... */                                 \
    ZPL_CONF_WAIT_DEFINE(name, threading)     \
    /* ... */
CONFIGS(ZPL_CONF_DEFINE)

// or invoke `CONFIGS` separately, e.g. in zpl/configuration_uart.c

#define ZPL_CONF_SHELL_DEFINE(name, ...) /* ... */
CONFIGS(ZPL_CONF_SHELL_DEFINE)
```

CHAPTER

FOUR

MEMORY PROFILING

The memory profiling functionality allows tracing of different memory regions. To use it, enable `CONFIG_ZPL_MEMORY_PROFILING` in the Kconfig.

4.1 Memory types

The profiling tool currently supports the following memory regions in Zephyr:

- Stack
- Heaps
- k_heaps
- Memory Slabs

4.2 Memory event

The memory tracing tool uses custom events. Those events contain information about the memory regions at a point in time.

4.2.1 Common Trace Format

The memory event is a packed structure, described below. It contains information about:

- `timestamp` - the timestamp at which the event occurred
- `id` - ID of the event defined as `0xEE`
- `memory_region` - the profiled memory region (stack, heap, etc.)
- `memory_addr` - the memory address, which serves as memory region identifier
- `used`, `unused` - used and unused number of bytes
- `for_thread_id` - ID of the thread associated with the memory region or 0 otherwise

```
typedef struct __packed {
    uint32_t timestamp;
    uint16_t id;
    enum zpl_memory_region memory_region;
    uintptr_t memory_addr;
    uint32_t used;
    uint32_t unused;
```

(continues on next page)

(continued from previous page)

```
    uint32_t for_thread_id;
} zpl_memory_event_t;
```

The available memory regions are defined as an enum:

```
enum zpl_memory_region {
    ZPL_STACK = 0,
    ZPL_HEAP,
    ZPL_K_HEAP,
    ZPL_MEM_SLAB,
};
```

4.2.2 Plaintext

When using a `CONFIG_ZPL_TRACE_FORMAT_PLAINTEXT` format, the memory events are printed in plaintext. The events contain the same information as in CTF but in human readable form. The example shown below contains a series of memory events of different memory regions.

- `zpl_memory_event` - event name
- `stack`, `heap`, etc. - memory region
- `0x20011748` - memory address
- `80B 432B` - used and unused memory size (in order)
- `536936848` - thread ID

```
zpl_memory_event stack (0x20011748) 80B 432B 536936848
zpl_memory_event stack (0x20011348) 48B 976B 536936848
zpl_memory_event stack (0x20012148) 48B 272B 536937024
zpl_memory_event stack (0x20012288) 280B 744B 536937200
zpl_memory_event heap (0x20010048) 28B 164B 0
zpl_memory_event heap (0x2001005c) 12B 940B 0
zpl_memory_event heap (0x200106ac) 0B 47132B 0
zpl_memory_event k_heap (0x20010048) 28B 164B 0
zpl_memory_event k_heap (0x2001005c) 12B 940B 0
zpl_memory_event mem_slab (0x2001002c) 256B 7936B 0
```

CHAPTER

FIVE

TRACING CODE SCOPES

Zephelin allows marking certain code scopes for tracing. When the program execution enters an enabled scope, an event is emitted before and after the code is executed. The scopes can be enabled and disabled at both build-time and runtime.

5.1 Marking the code

5.1.1 Defining the scopes

To use the scopes in your code, first define the scopes using the macro `ZPL_CODE_SCOPE_DEFINE`.

```
// ZPL_CODE_SCOPE_DEFINE(name, is_enabled);
ZPL_CODE_SCOPE_DEFINE(code_scope_name_1, false);
ZPL_CODE_SCOPE_DEFINE(code_scope_name_2, true);
```

The first parameter is the name of the code scope, which will be used as a handle for marking the code scopes. The second parameter, the flag `is_enabled`, describes the initial state of the scope. The scope can be enabled or disabled at boot, but it can always be switched either way at runtime.

5.1.2 Marking the code using `ZPL_MARK_CODE_SCOPE`

To mark the desired code scopes, Zephelin provides a special macro. The scope event will be emitted before and after the code inside is executed. The macro takes in one parameter, which is the name of the scope defined earlier using the macro `ZPL_CODE_SCOPE_DEFINE`.

```
ZPL_MARK_CODE_SCOPE(code_scope_name) {
    // ... Code inside the code scope
}
```

5.1.3 Marking the code using functions

Alternatively, you can mark the scopes using the enter/exit functions. Those functions work exactly the same as the macro showcased above.

```
zpl_code_scope_enter(code_scope_name);
// ... Code inside the code scope
zpl_code_scope_exit(code_scope_name);
```

This method allows for the entry and exit to be defined in separate contexts. For example, you can mark the beginning of a scope in one function, and the ending in a different function.

```
void start_function(void)
{
    zpl_code_scope_enter(code_scope_name);
    // Function body
}

void end_function(void)
{
    // Function body
    zpl_code_scope_exit(code_scope_name);
}
```

5.1.4 Example

In the example below, two scopes are created. Both methods (macro and function) are showcased. Marking the code scopes using both methods results in the same behavior.

```
#include <zpl/lib.h>

ZPL_CODE_SCOPE_DEFINE(code_scope1, false);
ZPL_CODE_SCOPE_DEFINE(code_scope2, true);

int main(void)
{
    ZPL_MARK_CODE_SCOPE(code_scope1) {
        // ... Code inside code_scope1
    }

    zpl_code_scope_enter(code_scope2);
    // ... Code inside code_scope2
    zpl_code_scope_exit(code_scope2);

    return 0;
}
```

5.2 Enabling/disabling code scopes at runtime

Code scopes can be enabled and disabled at runtime. This can be done in two ways: via UART shell, or by changing the scope's state in memory.

5.2.1 UART runtime configuration

To use the UART runtime configuration, enable the Kconfig option `CONFIG_ZPL_RUNTIME_CONFIG_UART`.

To list the available configs and their current state, use the command `dynamic_conf list`:

```
uart:~$ dynamic_conf list
Available configs:
 code_scope1: disabled
```

(continues on next page)

(continued from previous page)

```
code_scope2: disabled
code_scope3: enabled
code_scope4: enabled
```

To change the state of the scope, use `dynamic_conf enable <scope>` to enable and `dynamic_conf disable <scope>` to disable the scopes:

```
uart:~$ dynamic_conf enable code_scope1
uart:~$ dynamic_conf disable code_scope4
```

5.2.2 Debug runtime configuration

Code scopes can also be enabled directly in the memory, for example by using GDB. If the binary was build with debug symbols, then in GDB you can access them by name.

To check the state of a code scope, in GDB enter:

```
>>> print code_scope1.is_enabled
$1 = false
```

To change the state of a scope:

```
>>> set var code_scope1.is_enabled=1
>>> set var code_scope2.is_enabled=0
```

CHAPTER

SIX

TRACE CAPTURE USB INTERFACE REQUIREMENTS

This chapter describes the requirements to enable trace capture via Zephelin's USB interface. Zephelin provides an out-of-the-box USB device implementation in `zpl/backends/usb_backend.c`

To enable the USB interface, the user should y-select the `CONFIG_ZPL_TRACE_BACKEND_USB` Kconfig option. The `CONFIG_ZPL_TRACE_BACKEND_USB` option automatically selects every necessary software component in Zephyr required to run the USB device stack.

> **Note**
>
> Capturing traces via the USB backend requires asynchronous tracing to be enabled. To use it, y-select `CONFIG_TRACING_ASYNC`.

6.1 Driver requirements

A USB device driver needs to be provided for the given platform when the USB trace capture interface is used. Make sure there is a UDC driver available for your platform upstream (in `drivers/usb/udc/`), or provide your own UDC driver implementation as described in the Zephyr documentation.

6.2 Devicetree requirements

The USB device implemented by Zephelin expects the `zephyr_udc0` node to be present in the devicetree file. Make sure the platform devicetree file provides a valid `chosen` node with the `zephyr_udc0` property defined. The property under `/chosen/zephyr_udc0` should point to a platform USB controller node.

CHAPTER

SEVEN

TRACE CAPTURE DEBUG INTERFACE

Zephelin's debug interface enables for gathering traces via a GDB link. The traces are stored in memory and can be extracted using a `zpl-gdb-capture` west command.

Examples on how to capture traces, both on physical hardware and in Renode, are described below.

7.1 Physical hardware

Before capturing traces from a board, first make sure it's flashed with the firmware with the `CONFIG_ZPL_TRACE_BACKEND_DEBUGGER` Kconfig option enabled.

1. Connect the board via the debug interface to your host
2. Run the `zpl-gdb-capture` command and wait for the capture to finish

```
west zpl-gdb-capture <output_trace>
```

Note

The `zpl-gdb-capture` command runs the `west debugserver` command, so setting up the gdb server is not necessary. In order to provide different OpenOCD binary to `debugserver` command, please use `--openocd` argument.

7.2 Renode

Before capturing traces in Renode, first make sure firmware has been built with the `CONFIG_ZPL_TRACE_BACKEND_DEBUGGER` Kconfig option enabled.

1. Run Renode using `west simulate`
2. Enable the GDB server in Renode's Monitor: `machine StartGdbServer 3333`
3. Run the `zpl-gdb-capture` command and wait for the capture to finish

```
west zpl-gdb-capture --no-debug-server <output_trace>
```

7.3 Common options

By default, `zpl-gdb-capture` uses `gdb-multiarch` to connect with a debug server. To use a different GDB, provide it in the `--gdb` argument.

Moreover, the command dumps trace at the moment the connection with a debug server is established. This does not guarantee that data have been gathered, therefore one of two arguments can be used:

- `--buffer-full` - GDB runs an application and dumps the trace only when the buffer is full,
- `--n-bytes N` - GDB runs an application and dumps the trace only when the buffer has at least `N` bytes.

You can measure trace capturing speed by using `--measure-throughput`.

CHAPTER

EIGHT

INSTRUMENTATION

The instrumentation subsystem leverages the GCC feature to insert custom code right before and after a function calls for profiling and tracing applications.

8.1 Capturing trace

Currently, the instrumentation subsystem supports two tracing backends:

- UART
- Tracing subsystem

8.1.1 UART

It can be enabled with `CONFIG_INSTRUMENTATION_BACKEND_UART=y`.

When using the standalone UART backend, Zephelin provides the `west zpl-instrumentation-uart-capture` command to collect instrumentation traces:

```
west zpl-instrumentation-uart-capture [-h] \
     serial_port serial_baudrate output_path
```

It requests the trace by sending a `dump_trace` message and waits for a response. To make sure the received data is not contaminated by e.g. log messages, it is sent wrapped with `-*-#...-*-!`. Based on that, the command finds the beginning and the end of the message and saves the trace in CTF.

In case regular Zephelin traces are collected as well, it requires handling both trace channels separately.

8.1.2 Tracing subsystem

This backend can be enabled with `CONFIG_INSTRUMENTATION_BACKEND_TRACING_CORE=y`.

Instrumentation traces are packed together with regular traces in a separate CTF stream, allowing communication to be sent over a single channel, either UART, USB or GDB.

When using the tracing subsystem backend, instrumentation traces can be collected with the same `west` subcommand as regular traces, `zpl-uart-capture`:

```
 west zpl-uart-capture [-h] \
   serial_port serial_baudrate output_path
```

In order to visualize the trace with *Zephelin Trace Viewer*, it has to be converted using `west zpl-prepare-trace -i {INSTRUMENTATION_CTF_TRACE} {CONVERTED_TRACE}`. For more details, see *CTF to TEF conversion*.

8.2 Instrumentation with Zephelin's features

The instrumentation subsystem can also be used together with the rest of the Zephelin tracing systems.

The traces from the instrumentation subsystem are collected separately from the remaining tracing data. The requirements for collecting all data are following:

- A different backend or connection type for the instrumentation subsystem and the rest of traces - can be e.g. the UART backend for instrumentation and the RAM backend for other traces.
- The remaining traces being in CTF (configurable with `CONFIG_ZPL_TRACE_FORMAT_CTF=y`).

With this, the following `west` subcommand can be used to capture both instrumentation and Zephelin traces at the same time:

```
west zpl-instrumentation-uart-gdb-capture [-h] \
     serial_port serial_baudrate instr_output_path output_path
```

The command uses UART to capture the instrumentation trace (with the same logic as `zpl-instrumentation-uart-capture`) and GDB to extract remaining traces from RAM (as in `zpl-gdb-capture`). The produced files have to be converted to TEF and combined into one trace with `west zpl-prepare-trace -i {INSTRUMENTATION_TRACE} {ZEPHELIN_TRACE} {TEF_TRACE}`. The merging mechanism also ensures that traces do not collide and can be visualized.

8.3 Additional options

8.3.1 Running without retained memory

By default, the instrumentation subsystem uses retained memory in order to configure trigger and stopper functions via UART, and persistently store them as long as the device is powered.

The instrumentation subsystem can also be run without retained memory by disabling `CONFIG_INSTRUMENTATION_DYNAMIC_TRIGGER`. Then, trigger and stopper functions have to be selected with `CONFIG_INSTRUMENTATION_TRIGGER_FUNCTION` and `CONFIG_INSTRUMENTATION_STOPPER_FUNCTION` specifying mangled names of the functions.

8.3.2 Enabling and disabling instrumentation for Zephelin internals

A function can be excluded from the instrumentation in a few ways:

- `__no_instrumentation__` attribute added to the function definition,
- function name (not mangled) added to the `CONFIG_INSTRUMENTATION_EXCLUDE_FUNCTION_LIST`,
- file path containing the function code added to the `CONFIG_INSTRUMENTATION_EXCLUDE_FILE_LIST`.

Zephelin disables instrumentation of its internal functions by default to not pollute traces with events unrelated to the profiled application. It is achieved by adding source files to the exclude file list and using:

- `__no_zpl_instrumentation__` - extension of the `__no_instrumentation__` attribute that can be disabled with a custom config,
- `ZPL_DISABLE_INSTRUMENTATION` - macro disabling instrumentation for a given code scope, it is not compatible with mechanisms jumping out of the given code scope.

In order to enable instrumentation for Zephelin's internal functions the `CONFIG_ZPL_INTERNALS_INSTRUMENTATION` can be used. It prevents the source file from being added to the exclude list and disables both `__no_zpl_instrumentation__` and `ZPL_DISABLE_INSTRUMENTATION`.

Disabling instrumentation using the aforementioned options does not affect functions called from within excluded functions. To disable instrumentation of functions called within excluded function, enable `CONFIG_INSTRUMENTATION_RECURSIVE_EXCLUDE` and list function names in `CONFIG_INSTRUMENTATION_RECURSIVE_EXCLUDE_LIST`. Ensure that functions listed in `CONFIG_INSTRUMENTATION_RECURSIVE_EXCLUDE_LIST` are not excluded using the non-recursive methods (e.g. `__no_instrumentation__`), as this will prevent the recursive exclusion flags from being set.

The instrumentation subsystem provides a list of default excludes designed to prevent the instrumentation of the instrumentation and tracing subsystems. Before configuring custom excludes, consider appending them to the defaults.

8.3.3 Dumping events when the instrumentation buffer is full

Instead of disabling the instrumentation subsystem when the buffer is filled or overwriting the existing data, the subsystem can also dump all events to make space in the buffer. This option can be enabled with `CONFIG_INSTRUMENTATION_MODE_CALLGRAPH_DUMP_ON_FULL=y`.

The `zpl-instrumentation-uart-capture` and `zpl-instrumentation-uart-gdb-capture` commands can automatically detect whether `DUMP_ON_FULL` was used and adjust the capturing mechanism accordingly. Similarly to Zephelin, the instrumentation subsystem sends an init tag at the start of the application. It is detected by the `west` commands and saves traces into a separate file, therefore it is advised to start the `zpl-instrumentation-uart-capture` command before flashing or restarting the board. Next, the commands wait for binary messages until the user stops the process or there are no new messages received in a specified timeout. To make sure all events are captured, after the timeout, the commands send `dump_trace` and gather the remaining data from the buffer.

An example of this mechanism can be found in samples/profiling/tflm_instrumentation.

8.3.4 Separating instrumentation traces from regular traces

To visually separate instrumentation traces from regular traces in *Zephelin Trace Viewer*, the `--separate-instr-pid` option can be passed to `west zpl-prepare-trace`. Instrumentation traces will be placed on a separate profile with the name matching the original thread name but with the `(instrumentation)` suffix added. Using this option avoids the need to tamper with event timestamps to align both types of traces.

CHAPTER

NINE

CTF TO TEF CONVERSION

In order to expand the compatibility of the traces received in Common Trace Format (CTF), we created a converter for obtaining them in the Trace Event Format (TEF).

9.1 Usage

You can run the converter with the following command:

```
usage: west zpl-prepare-trace \
  [-h] -o OUTPUT [--zephyr-base ZEPHYR_BASE] \
  [--tflm-model-paths TFLM_MODEL_PATHS [TFLM_MODEL_PATHS ...]] [--tflm-model-ids␣
↪TFLM_MODEL_IDS [TFLM_MODEL_IDS ...]] \
  [--tvm-model-paths TVM_MODEL_PATHS [TVM_MODEL_PATHS ...]] [--tvm-model-metadata-
↪paths TVM_MODEL_METADATA_PATHS [TVM_MODEL_METADATA_PATHS ...]] \
  [--tvm-model-op-remove-prefix TVM_MODEL_OP_REMOVE_PREFIX] [--tvm-model-op-
↪remove-suffix TVM_MODEL_OP_REMOVE_SUFFIX] \
  [--build-dir BUILD_DIR] [--zephyr-elf-path ZEPHYR_ELF_PATH] [-i␣
↪INSTRUMENTATION] \
  ctf_trace
```

There are several optional arguments available:

- `--zephyr-base` - the script tries to automatically find the Zephyr directory in order to use the metadata file definition of CTF events. If this fails, you have to provide the path or set the `ZEPHYR_BASE` environmental variable.
- `--build-dir` - the directory storing the results of the build and ram_report.
- `--zephyr-elf-path` - the path to the generated Zephyr ELF file, it should be deduced based on the build directory.
- `--tflm-model-path` - the provided TFLite Micro model is processed to extract information about its layer and tensors, and the information is converted to a `MODEL` Metadata Event.
- `--tflm-model-ids` - the IDs of TFLM models, the order has to be the same as in `--tflm-model-path`. It has to be used only if the default mechanism deducing IDs based on the models' addresses fails.
- `--tvm-model-path` - the provided microTVM graph JSON file is processed to extract information about the model's layer and tensors, and the information is converted to a `MODEL` Metadata Event.
- `--tvm-model-metadata-path` - the provided microTVM Intermediate Representation file is processed to extract more information about the operator parameters. It can be ob-

tained from the compiled module. (see scripts/export_tvm_metadata.py for reference; please note that `--tvm-module-name` has to be specified if a custom module name was used, like `quantized` for the sine model, based on the generated functions' prefixes from samples/common/tvm/src/sine.h). Verified TVM versions: `v0.14.dev273` - `v0.18.0`.

- `--tvm-model-op-remove-prefix` - the provided regular expression is used to remove the microTVM operator prefixes. Applied on both operator types and instances. The default value is `^tvmgen_default_`.
- `--tvm-model-op-remove-suffix` - the provided regular expression is used to remove the microTVM operator type suffixes. Applied on operator types. The default value is `_\d*$`.
- `-i` or `--instrumentation` - the option to provide traces produced by the instrumentation subsystem.
- `ctf_trace` - the path to the trace produced by Zephelin, the script requires at least one trace either from Zephelin or the instrumentation.
- `-o` - the file path which points to the file where the converted trace should be saved. If not provided, the JSON will be printed to STDOUT.

The extension uses the `c++filt` utility to demangle function names by default. A custom tool can be specified by setting the `ZPL_DEMANGLE_CMD` environment variable.

Apart from changing the format, this involves a custom logic which can group elements into Duration events, and extend their arguments (see *Converted events*). Completely new events can be added to the trace, extending the context and improving the trace visualization capabilities (see *Additional events*). Furthermore, the command also converts timestamps from nanoseconds (used in CTF) to microseconds, which are valid for Zephelin Trace Viewer and Speedscope (it assumes that the timestamps are provided in such unit).

When two traces are provided, from Zephelin and the instrumentation, the script will try to combine them. If the instrumentation trace has unfinished events, they will be automatically closed ensuring they do not collide with Zephelin events. Events without a beginning will be removed.

9.2 Events

9.2.1 Converted events

This section describes events that are converted from CTF events emitted during Zephelin runtime. Based on those, the trace viewer produces dedicated panels with visualizations (e.g. plot panels).

Zephyr events

For default Zephyr events (defined in metadata), the beginning event always gets the `_enter` suffix, whereas the ending event gets the `_exit` suffix. These events are marked as `B` and `E` events, creating the Duration event in TEF. The remaining Zephyr events are of the Instant type, but since Speedscope does not support displaying such events, they are also converted to a Duration Event with 1 microsecond of duration.

Example events

```
{
  "name": "isr",
  "cat": "zephyr",
  "ph": "B",
  "ts": 99967.43400000001,
  "pid": 0,
  "tid": 0
},
"...",
{
  "name": "isr",
  "cat": "zephyr",
  "ph": "E",
  "ts": 99980.142,
  "pid": 0,
  "tid": 0
}
```


INFERENCE::MODEL[NUMBER]

The Duration event marking the beginning and end of model's inference. It is created from the CTF events `zpl_inference_enter` and `zpl_inference_exit` and creates the `INFERENCE::MODEL[NUMBER]` event containing the ID of the model that is executed. `NUMBER` is an optional model number (assigned based on the model ID starting from 0), shown only when the trace contains more than one model.

zpl_inference events examples


```
{
  "name": "INFERENCE::MODEL2",
  "cat": "zephyr",
  "ph": "B",
  "ts": 5582.45,
  "pid": 0,
  "tid": 536893200,
  "args": {
    "thread_id": 536893200,
    "model_id": 268506268
  }
},
"...",
{
  "name": "INFERENCE::MODEL2",
  "cat": "zephyr",
  "ph": "E",
  "ts": 393496.442,
  "pid": 0,
  "tid": 536893200,
  "args": {
    "thread_id": 536893200,
```

```
    "model_id": 268506268
  }
}
```

MODEL[NUMBER]::{LAYER_OP}[_{SUBGRAPH_IDX}]_{OP_IDX}

The MODEL[NUMBER]::* Duration event is unique to each layer of a model, described with:

- NUMBER - an optional model number - the same as in *INFERENCE::MODEL[NUMBER]*, shown only when more than one model is present in the trace,
- LAYER_OP - a tag of operation like CONV_2D or MAX_POOL_2D,
- SUBGRAPH_IDX - an optional number representing the ID of a subgraph to which a given layer belongs,
- OP_IDX - a number representing the ID of the operation in a subgraph.

The event contains:

- identifiers like op_idx and optional subgraph_idx,
- tag with the layer name,
- thread_id pointing to the thread that executed this layer,
- runtime specifying which runtime was used, and
- additional runtime-specific data.

This event is converted from TFLite micro (zpl_tflm_(enter|exit)) and microTVM (zpl_tvm_(enter|exit)) events.

MODEL::* events examples

```
{
  "name": "MODEL2::CONV_2D_0_0",
  "cat": "zephyr",
  "ph": "B",
  "ts": 4294973.212,
  "pid": 0,
  "tid": 536912424,
  "args": {
    "thread_id": 536912424,
    "subgraph_idx": 0,
    "op_idx": 0,
    "tag_len": 20,
    "tag": "CONV_2D",
    "arena_used_bytes": 15408,
    "arena_tail_usage": 88,
    "runtime": "TFLite Micro"
  }
},
{
  "name": "MODEL2::CONV_2D_0_0",
```

```
  "cat": "zephyr",
  "ph": "E",
  "ts": 4359202.146,
  "pid": 0,
  "tid": 536912424,
  "args": {
    "thread_id": 536912424,
    "subgraph_idx": 0,
    "op_idx": 0,
    "tag_len": 20,
    "tag": "CONV_2D",
    "arena_used_bytes": 15408,
    "arena_tail_usage": 88,
    "runtime": "TFLite Micro"
  }
}
```


MEMORY

The Metadata event with information about a *memory region* for a given timestamp (`ts`). It is created from the CTF event `zpl_memory`.

MEMORY metadata example


```
{
  "name": "MEMORY",
  "cat": "zephyr",
  "ph": "M",
  "ts": 55.216,
  "pid": 0,
  "tid": 536937200,
  "args": {
    "memory_region": "STACK",
    "memory_addr": 536950376,
    "used": 88,
    "unused": 424,
    "for_thread_id": 536936848
  }
}
```


CPU_LOAD

The Metadata event defining CPU load (`cpu_load` field) for a given timestamp (`ts`). It is converted from the CTF event `zpl_cpu_load_event`.

CPU_LOAD metadata example


```
{
  "name": "CPU_LOAD",
```

```
  "cat": "zephyr",
  "ph": "M",
  "ts": 200044.308,
  "pid": 0,
  "tid": 0,
  "args": {
    "cpu_load": 534
  }
}
```


DIE_TEMP

The Metadata event providing die temperatures (`die_temp` array with at most two measurements, in degrees Celsius) for a given timestamp (`ts`). It is converted from the CTF event `zpl_die_temp_event`.

DIE_TEMP metadata example


```
{
  "name": "DIE_TEMP",
  "cat": "zephyr",
  "ph": "M",
  "ts": 300327.025,
  "pid": 0,
  "tid": 0,
  "args": {
    "die_temp": [
      21.947092056274414,
      41.94709014892578
    ]
  }
}
```


9.2.2 Additional events

This section contains events that are not produced during the Zephelin runtime, but are used to provide additional information for better visualizations.

MEMORY::STATICALLY_ASSIGNED_MEM

The Metadata event informing about the part of RAM (in bytes), used by the compiled objects. This value is calculated using size from ram_report and subtracting sizes of the memory regions (from MEMORY *events*). The event looks like this:

MEMORY::STATICALLY_ASSIGNED_MEM metadata example


```
{
  "name": "MEMORY::STATICALLY_ASSIGNED_MEM",
  "cat": "zephyr",
```

```
  "ph": "M",
  "pid": 0,
  "tid": 0,
  "ts": 0,
  "args": 14105
}
```

MEMORY::SYMBOLS

The Metadata event contains the mapping of memory region addresses to their symbols extracted from `zephyr.elf`. It is used to present a human-readable description of the regions, e.g. making it easier to trace back to the source code. Example event:

MEMORY::SYMBOLS metadata example

```
{
  "name": "MEMORY::SYMBOLS",
  "cat": "zephyr",
  "ph": "M",
  "pid": 0,
  "tid": 0,
  "ts": 0,
  "args": {
    "536950376": "_k_thread_stack_zpl_mem_profiling",
    "536949352": "tracing_thread_stack",
    "536952936": "z_idle_stacks",
    "536953256": "z_main_stack",
    "536938172": "z_malloc_heap",
    "536936540": "_system_heap",
    "536936560": "test_k_heap",
    "536936512": "test_mem_slab"
  }
}
```

thread_name

The Metadata event is used by Speedscope to associate a thread ID with a name. Apart from describing which thread is shown on the flamegraph, the thread name is also used to provide a human-readable label for the thread stack.

One event describes exactly one thread with arguments containing the thread name, assigned to the key `name`.

thread_name metadata example

```
{
  "name": "thread_name",
  "cat": "zephyr",
  "ph": "M",
```

```
  "pid": 0,
  "tid": 536912424,
  "args": {
    "name": "main"
  }
}
```

MODEL

The `MODEL` Metadata event contains following information:

- `inputs` - the specification of model inputs, described with `name`, `shape` and `dtype`,
- `outputs` - the specification of model outputs, with the same properties as `inputs`,
- `tensors` - the specification of internal data like inputs and outputs of layers, their weights or biases; the properties are the same as `inputs`, but with additional `index` and optional `subgraph_idx`,
- `ops` - the specification of model operations:
  - `op_name` - the name of the operation (e.g. `CONV_2D` or `tvmgen_default_fused_nn_conv2d_add_nn_relu`),
  - `index` - the ID of the operation,
  - `inputs` and `outputs` - lists with indices pointing to `tensors` with input and output data (respectively),
  - `inputs_types` and `outputs_types` - the operation's input and output data types,
  - `inputs_shapes` and `outputs_shapes` - the operation's input and output shapes,
  - `parameters` - the operation's parameters (e.g. `padding` or `fused_activation_function`).

Example event of Magic Wand model for TFLite Micro runtime

```
{
  "name": "MODEL",
  "cat": "zephyr",
  "ph": "M",
  "pid": 0,
  "tid": 0,
  "ts": 0,
  "args": {
    "inputs": [
      {
        "name": "input_1",
        "name_long": "serving_default_input_1:0",
        "shape": [1, 128, 3, 1],
        "shape_signature": [-1, 128, 3, 1],
        "dtype": "float32",
        "quantization": [0.0, 0],
        "quantization_parameters": {
```

```
          "scales": [],
          "zero_points": [],
          "quantized_dimension": 0
        }
      }
    ],
    "outputs": [
      {
        "name": "out_layer",
        "name_long": "StatefulPartitionedCall:0",
        "shape": [1, 4],
        "shape_signature": [-1, 4],
        "dtype": "float32",
        "quantization": [0.0, 0],
        "quantization_parameters": {
          "scales": [],
          "zero_points": [],
          "quantized_dimension": 0
        }
      }
    ],
    "tensors": [
      {
        "name": "serving_default_input_1:0",
        "subgraph_idx": 0,
        "index": 0,
        "shape": [1, 128, 3, 1],
        "shape_signature": [-1, 128, 3, 1],
        "dtype": "float32",
        "quantization": [0.0, 0],
        "quantization_parameters": {
          "scales": [],
          "zero_points": [],
          "quantized_dimension": 0
        }
      },
      "..."
    ],
    "ops": [
      {
        "op_name": "CONV_2D",
        "index": 0,
        "inputs": [0, 3, 1],
        "outputs": [10],
        "inputs_types": ["float32", "float32", "float32"],
        "outputs_types": ["float32"],
        "inputs_shapes": {
          "0": [1, 128, 3, 1],
          "3": [8, 4, 3, 1],
          "1": [8]
```

```
        },
        "outputs_shapes": {
          "10": [1, 128, 3, 8]
        }
      },
      "..."
    ]
  }
}
```

CHAPTER

TEN

ADDING SUPPORT FOR NEW AI INFERENCE LIBRARIES

This chapter describes a few ways to trace a new AI runtime:

- *Using dedicated runtime events* - requires additional steps, but provides the most complex information and visualization,
- *Using instrumentation subsystem* - can be used as is, but does not provide all the necessary information, therefore making some visualization unavailable.

10.1 Implementing support

A runtime support provides versatile information about a model's execution and its architecture, helping to analyze and optimize an application. This section describes how to implement the support based on a TFLM profiling example.

10.1.1 Create new events for the runtime

First and foremost, a new runtime-specific events have to be defined. As the CTF (Common Trace Format) format requires an additional metadata file with the event structures, all definitions are placed in zpl/metadata. In this file, `zpl_tflm_enter` and `zpl_tflm_exit` are defined:

```
event {
        name = zpl_tflm_enter;
        id = 0xA0;
        fields := struct {
                uint32_t thread_id;
                uint16_t subgraph_idx;
                uint16_t op_idx;
                uint16_t tag_len;
                ctf_bounded_string_t tag[tag_len];
                uint32_t arena_used_bytes;
                uint32_t arena_tail_usage;
        };
};

event {
        name = zpl_tflm_exit;
        id = 0xA1;
        fields := struct {
                uint32_t thread_id;
                uint16_t subgraph_idx;
```

(continues on next page)

(continued from previous page)

```
                uint16_t op_idx;
                uint16_t tag_len;
                ctf_bounded_string_t tag[tag_len];
                uint32_t arena_used_bytes;
                uint32_t arena_tail_usage;
        };
};
```

There are several things to keep in mind when adding new events:

- Each event inherits the `id` and `timestamp` fields from the header defined in Zephyr,
- The `id` has to be unique across the given model,
- Two events are defined to represent a beginning and an end of an operation,
- A runtime event has to have:
  - `op_idx` - ID of the operation instance in the graph,
  - `thread_id` - ID of the thread executing the operation,
  - `tag` - name of the operation (its length can be hard-coded, or dynamic based on another parameter - `tag_len` in this example).
- `subgraph_idx` is an optional field used if a model is subdivided into several subgraphs (e.g. executed on different processors),
- Other parameters of the event are not required, but will be displayed by the Trace Viewer.

The same structure has to be mirrored in the code (a full example can be found in include/zpl/tflm_event.h):

```
/* TFLite Micro events IDs */
#define ZPL_TFLM_ENTER_EVENT 0xA0
#define ZPL_TFLM_EXIT_EVENT 0xA1

typedef struct __packed {
  // Event header defined by Zephyr
        uint32_t timestamp;
        uint16_t id;
        // Event fields
        uint32_t thread_id;
        uint16_t subgraph_idx;
        uint16_t op_idx;
        uint16_t tag_len;
        uint8_t tag[CONFIG_ZPL_TRACE_CTF_MAX_STR_LEN];
        uint32_t arena_used_bytes;
        uint32_t arena_tail_usage;
} zpl_tflm_event_t;
```

The struct has to define the same fields with the same types, as well as `timestamp` and `id` from the event header. Moreover, the `__packed` attribute is required to avoid gaps between data, which would break the CTF structure.

Using this, events can be emitted with the trace_format_raw_data function, e.g.:

```
#include <zephyr/tracing/tracing_format.h>

zpl_tflm_event_t zpl_tflm_exit_event = {
        .timestamp = zpl_timestamp_get_from_cycles(cycles),
        .id = is_exit ? ZPL_TFLM_EXIT_EVENT : ZPL_TFLM_ENTER_EVENT,
        .thread_id = (uint32_t)k_current_get(),
        .subgraph_idx = subgraph_idx,
        .op_idx = op_idx,
        .arena_used_bytes = arena_used_bytes,
        .arena_tail_usage = arena_tail_usage,
        .tag_len = CONFIG_ZPL_TRACE_CTF_MAX_STR_LEN,
};

tracing_format_raw_data(
        (uint8_t *)&zpl_tflm_exit_event, sizeof(zpl_tflm_exit_event)
);
```

The example shows how to emit events in CTF; for human-readable format, the TRACE_STRING macro can be used:

```
TRACING_STRING(
        "zpl_tflm_%s_event: subgraph_idx=%d op_idx=%d tag=%s "
        "arena_used_bytes=%d arena_tail_usage=%d\n",
        is_exit ? "exit" : "enter", subgraph_idx, op_idx, tag,
        arena_used_bytes, arena_tail_usage);
```

The two approaches should be combined to easily change a trace format. It can be achieved by selecting the code with the `ifdef` directive checking the `CONFIG_ZPL_TRACE_FORMAT_CTF` or `CONFIG_ZPL_TRACE_FORMAT_PLAINTEXT` options. An example can be found in zpl/profilers/tflm/tflm_event.c.

It is advised to wrap the emitting mechanism into one function for easier use.

10.1.2 Emit events in the runtime

The next step is to emit the new events from the runtime. This can differ greatly based on the runtime's architecture, but it is suggested to emit events right before and after an operation is executed, ensuring the precision of the trace.

One way to achieve that is to use a callback-based approach - implementing event emitting functions or methods that will be automatically called by the runtime. For instance, in TFLM, such methods are available in the `MicroProfilerInterface` class (`BeginEvent` and `EndEvent`).

Optionally, instead of emitting events right away in the callback, they can be stored in the buffer and emitted after an inference is finished. This decreases tracing overhead on the inference times.

The implementation for TFLM can be found in zpl/profilers/tflm/tflm_profiler.cpp.

Moreover, it is advised to emit the `zpl_inference_enter` and `zpl_inference_exit` events, respectively before and after an inference, in order to keep track of the whole model inference time. To emit them, functions from include/zpl/inference_event.h can be used.

The `zpl_inference` events require information about the model ID, which is used in scenarios when more than one model is executed. In case of TFLM, the model address is used as the ID.

> **Note**
>
> A similar effect can be achieved with *Tracing code scopes*, but this approach is not as extensible as the one described above, making it harder to work with.

10.1.3 Add custom processing for new events

In order to use new events as the ones describing model inference, they have to be converted to *MODEL[NUMBER]::{LAYER_OP}[_{SUBGRAPH_IDX}]_{OP_IDX}* TEF events. It can be done with a custom event processing during conversion from CTF to TEF, defined in scripts/prepare_trace. A custom event definition has to be returned by the `create_custom_events` function.

```
def tflm_op_name(msg: "bt2._EventMessageConst") -> str:
    fields = msg.event.payload_field
    if not fields:
        return ""
    name = str(fields.get("tag", ""))

    # Add subgraph index at the end, if exists
    if "subgraph_idx" in fields:
        name += f"_{fields['subgraph_idx']}"

    # Add operator index at the end
    name += f"_{fields['op_idx']}"

    return name

CustomEventDefinition(
    "MODEL::",
    "zpl_tflm_enter",
    "zpl_tflm_exit",
    tflm_op_name,
    lambda _: {"runtime": "TFLite Micro"},
)
```

This example specifies that `zpl_tflm_enter` and `zpl_tflm_exit` will be converted to the Duration event - *MODEL[NUMBER]::{LAYER_OP}[_{SUBGRAPH_IDX}]_{OP_IDX}*, where the suffix of the event name will created by the `tflm_op_name` function. Moreover, custom events will be appended with an additional parameter produced by the last function, in this case `runtime="TFLite Micro"`.

The custom event name has to be unique for each operation type, therefore it is advised to use the `op_idx` field. Furthermore, to improve the readability of trace, it should also contain a human-readable name (e.g. `tag` field).

10.1.4 Implement model metadata extraction

You can provide more information about the model with *MODEL*. Based on the information provided, the plot with the operator size will be available, together with extra properties in the `Detail` view.

In case of TFLM, metadata are extracted from a model with LiteRT's `Interpreter` and FlatBuffer schema - see scripts/extract_tflite_model_data.py.

Furthermore, this mechanism can be easily integrated with the `west zpl-prepare-trace` command by appending a *MODEL* event after the conversion, in scripts/prepare_trace.py, like this:

```
if args.tflm_model_path is not None:
    from extract_tflite_model_data import extract_model_data

    add_model_metadata(tef_trace, extract_model_data(args.tflm_model_path, args.
↪zephyr_base))
```

If the runtime will be used with multiple models at once, it is suggested to prepare the mechanism that will deduce IDs of the models automatically. Otherwise, users will have to provide them manually during conversion. For TFLM, it is done by finding model data in Zephyr ELF file and offsetting it by flash region address. As this method does not work when model data are not present in the flash, there is an additional parameter to provide the IDs manually - see `--tflm-model-ids` of `west zpl-prepare-trace` command.

On the other hand, models can also be differentiate based on the event arguments. For microTVM, it can be achieved by including the module name in the `tag` argument of `MODEL` *event* - each function starting with `tvmgen_{MODULE_NAME}_`. Based on that, `INFERENCE` *events* are updated to contain the model ID associated with the module name (see `tvm_recalculate_model_numbers` in scripts/extract_tvm_model_data.py). The model ID can be matched to the model metadata in the same way.

10.2 Instrumentation subsystem

The instrumentation subsystem allows to automatically trace entering and exiting functions, which in most runtimes can be used to trace the execution of the model and its layers.

> **Note**
>
> Depending on the granularity of the runtime's functions, the received trace can contain between one event per model inference and multiple events per layer.

Based on the TFLM instrumentation sample, the subsystem can be enabled with:

```
# Enables the instrumentation subsystem
CONFIG_INSTRUMENTATION=y
# Uses the trace buffer as a normal buffer, instead of a ring buffer
CONFIG_INSTRUMENTATION_MODE_CALLGRAPH_BUFFER_OVERWRITE=n
# (Optional) Sets the size of the buffer
CONFIG_INSTRUMENTATION_MODE_CALLGRAPH_TRACE_BUFFER_SIZE=51200
# (Optional) Disables dynamic trigger/stopper functions configuration, so that␣
```

(continues on next page)

(continued from previous page)

```
↪retained memory is not required
CONFIG_INSTRUMENTATION_DYNAMIC_TRIGGER=n
```

Note

The instrumentation subsystem supports only traces in CTF, so there is no need to define the `ZPL_TRACE_FORMAT` config.

To focus at a specific part of the application, a trigger and stopper function can be set to capture a trace only within selected scope. For instance, in order to trace the main inference loop of a TFLM instrumentation sample, its symbol has to be extracted from the Zephyr ELF (e.g. with `nm` or `objdump`). Then, after the board is flashed, the trigger and stopper can be set with the `zaru` script:

```
# Sets main loop as a trigger and stopper
${ZEPHYR_BASE}/scripts/instrumentation/zaru.py --serial ${UART_PORT} trace -c $
↪{LOOP_SYMBOL}
# Reboot the board, to capture the trace with selected
${ZEPHYR_BASE}/scripts/instrumentation/zaru.py --serial ${UART_PORT} reboot
```

Currently, traces from the instrumentation subsystem can only be captured via UART, with a separate command:

```
usage: west zpl-instrumentation-uart-capture [-h]
                                             serial_port serial_baudrate
                                             output_path

Capture instrumentation traces using UART. This command captures traces using
the serial interface.

positional arguments:
  serial_port      Seral port
  serial_baudrate  Seral baudrate
  output_path      Capture output path
```

Moreover, when converting the trace to TEF, `west zpl-prepare-trace` has to be used with the `--instrumentation` flag:

```
west zpl-prepare-trace -o ${TEF_TRACE} --instrumentation ${CTF_TRACE}
```

Note

Instrumentation traces can also be forwarded through the *tracing subsystem*, which has broader backend support.

The resulting TEF trace can be visualized with *Zephelin Trace Viewer*.

The instrumentation subsystem does not filter out not runtime-related events, therefore the

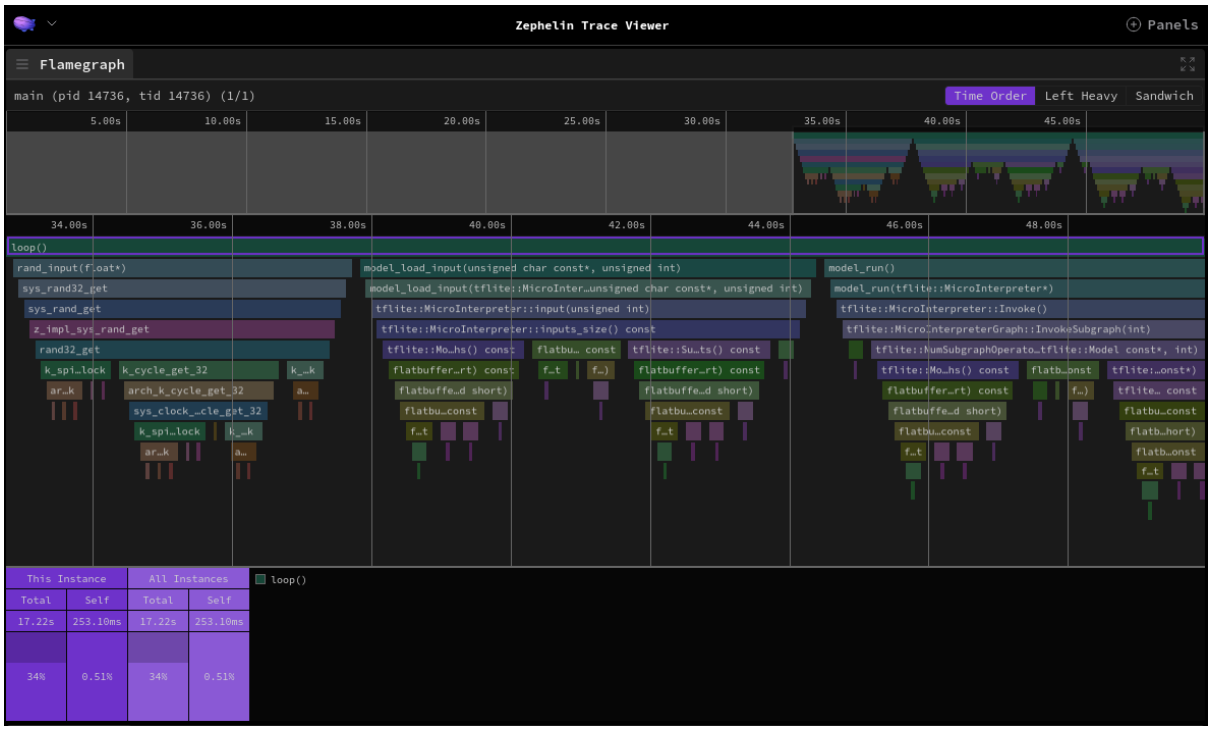


Fig. 10.1: The trace from the instrumentation subsystem executing the main inference loop of TFLM runtime

trace will contain a lot of data. Despite that, the number of available runtime-related visualizations will be restricted to only flamegraphs.

CHAPTER

ELEVEN

ZEPHELIN TRACE VIEWER

Zephelin Trace Viewer is a visual interface for traces in Trace Event Format (TEF). To fully utilize the features of the viewer, we suggest processing traces with the *CTF converter*.

Go to the GitHub-hosted Zephelin Trace Viewer to begin analyzing traces.

11.1 Requirements

The viewer leverages workspaces to manage third-party dependencies like Speedscope. This requires yarn in version 1.x or newer, provided by corepack (usually installed together with Node.js):

```
# Allows to use package managers without having to install them
corepack enable
# Downloads specified version of yarn and all dependencies
yarn
```

11.2 Building

There are several predefined commands with basic utilities:

- `yarn build` - Builds for production, emitting to `dist/`
- `yarn preview` - Starts a server at `http://localhost:4173/` to test production build locally
- `yarn dev` - Starts a dev server at `http://localhost:5173/`
- `yarn lint` - Lints the code and applies all applicable fixes

11.3 Usage

The viewer is built around Speedscope - an interactive flamegraph viewer with support for many trace/profiling formats.

To display a trace, simply import a TEF file using the `File/Import trace` (or `Browse`) button (Fig. 11.1). This action will change the internal state of Speedscope, which triggers the reaction of the `Additional info` section. It gathers metadata and initializes custom panels based on available data.

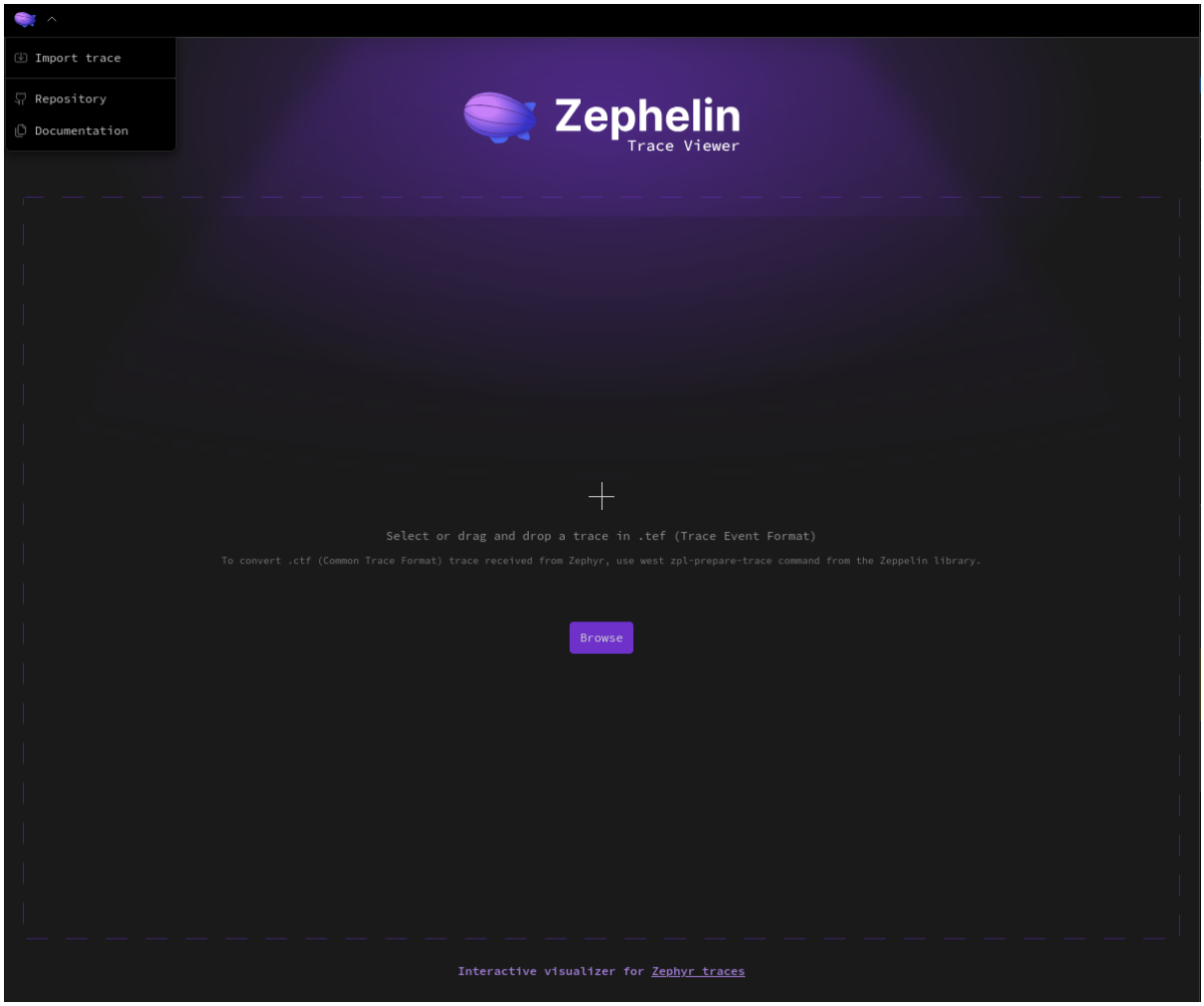


Fig. 11.1: The main page of Trace Viewer

11.3.1 Model info panel

The model info panel displays details about the selected event and the corresponding layer.

Currently, it supports TFLite Micro (Fig. 11.2)

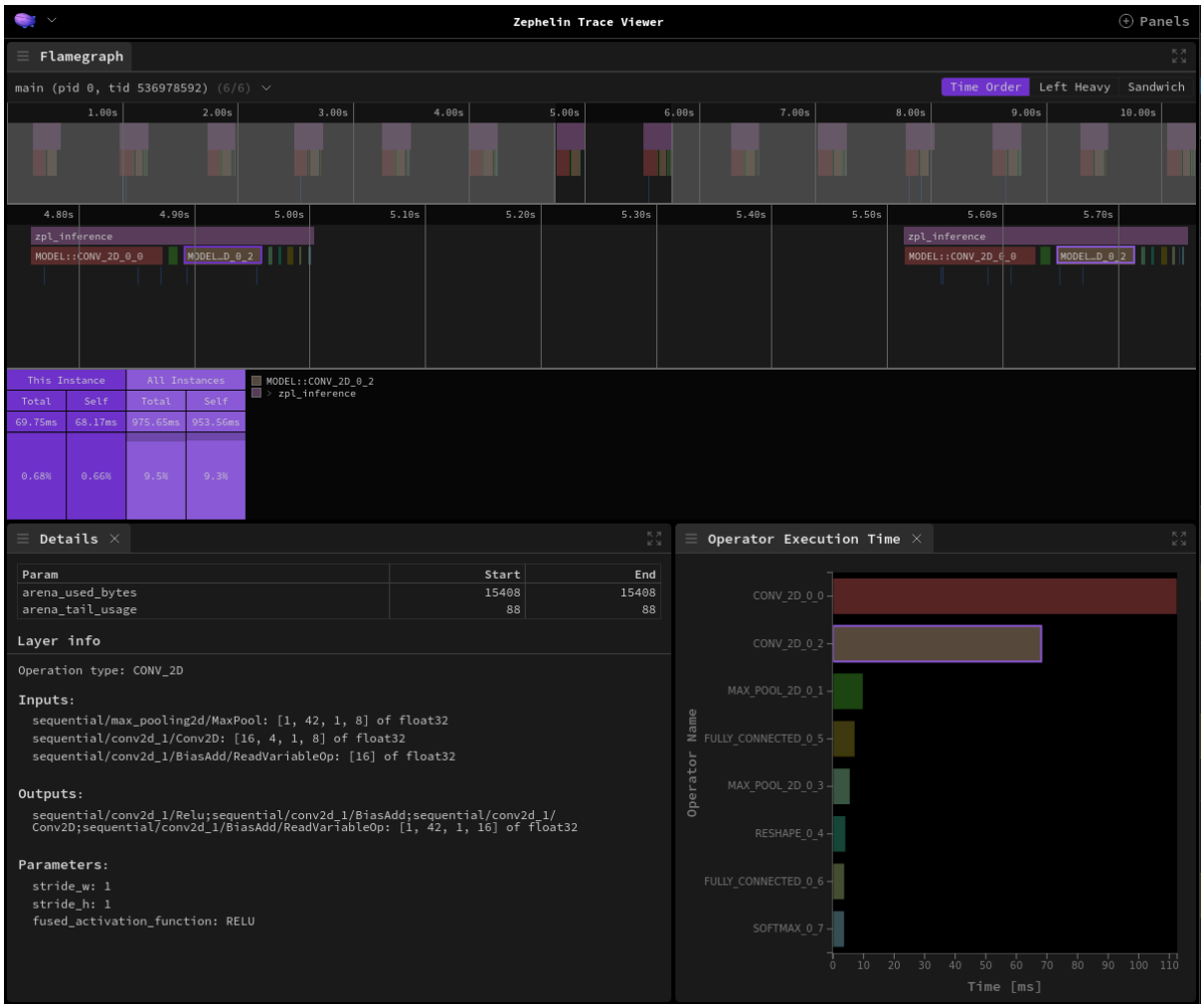


Fig. 11.2: The TFLite micro model information example

and microTVM runtime (Fig. 11.3).

11.3.2 Plot panels

The Trace Viewer can also visualize resource usage (like memory or CPU load) or data from sensors (such as die temperature) using plots (Fig. 11.4).

Panels with these plots are automatically created whenever a trace with the necessary data is loaded:

- *MEMORY* events for memory plots,
- *CPU_LOAD* events for CPU load plot,
- *DIE_TEMP* events for die temperature plot.

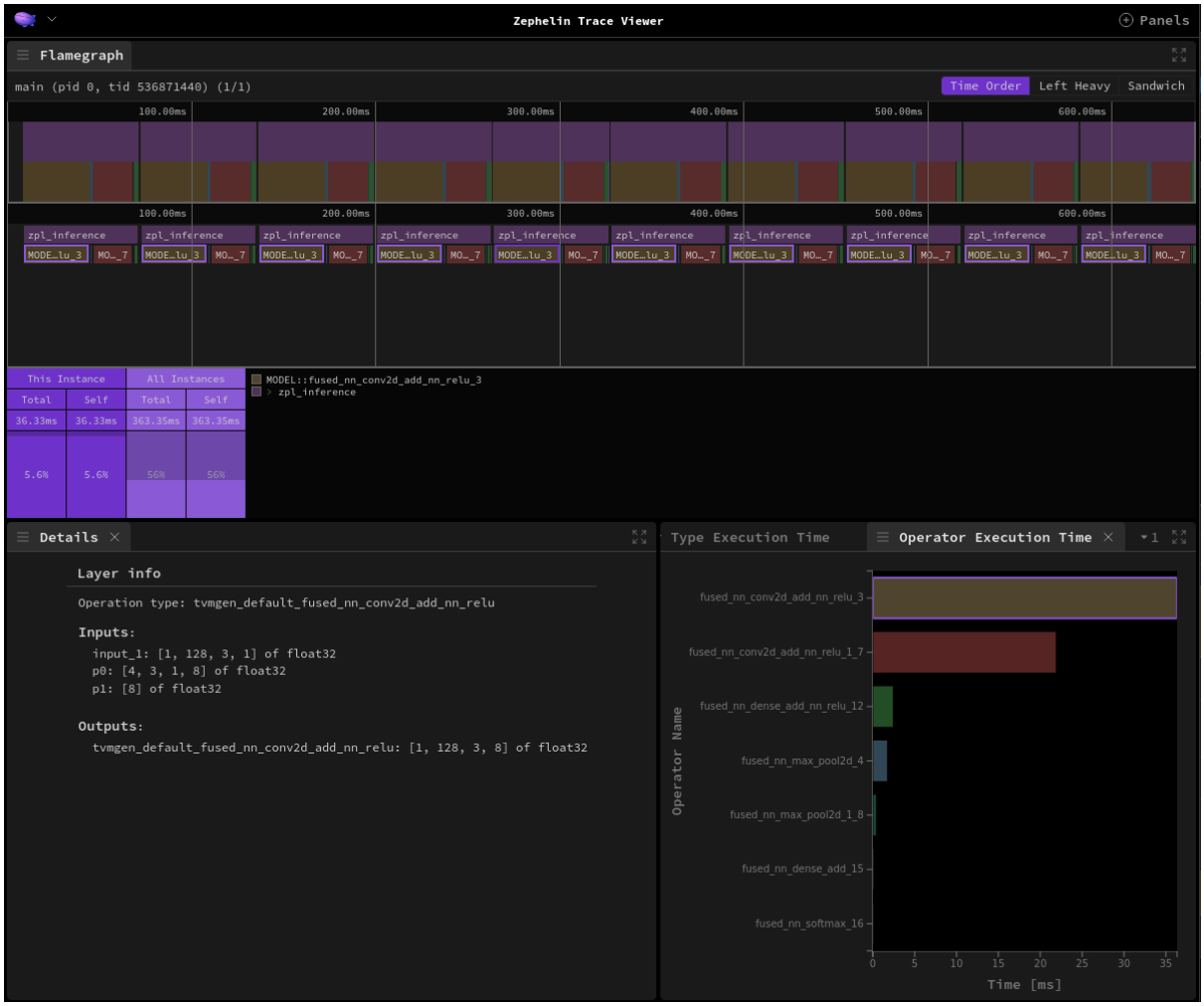


Fig. 11.3: The microTVM model information example

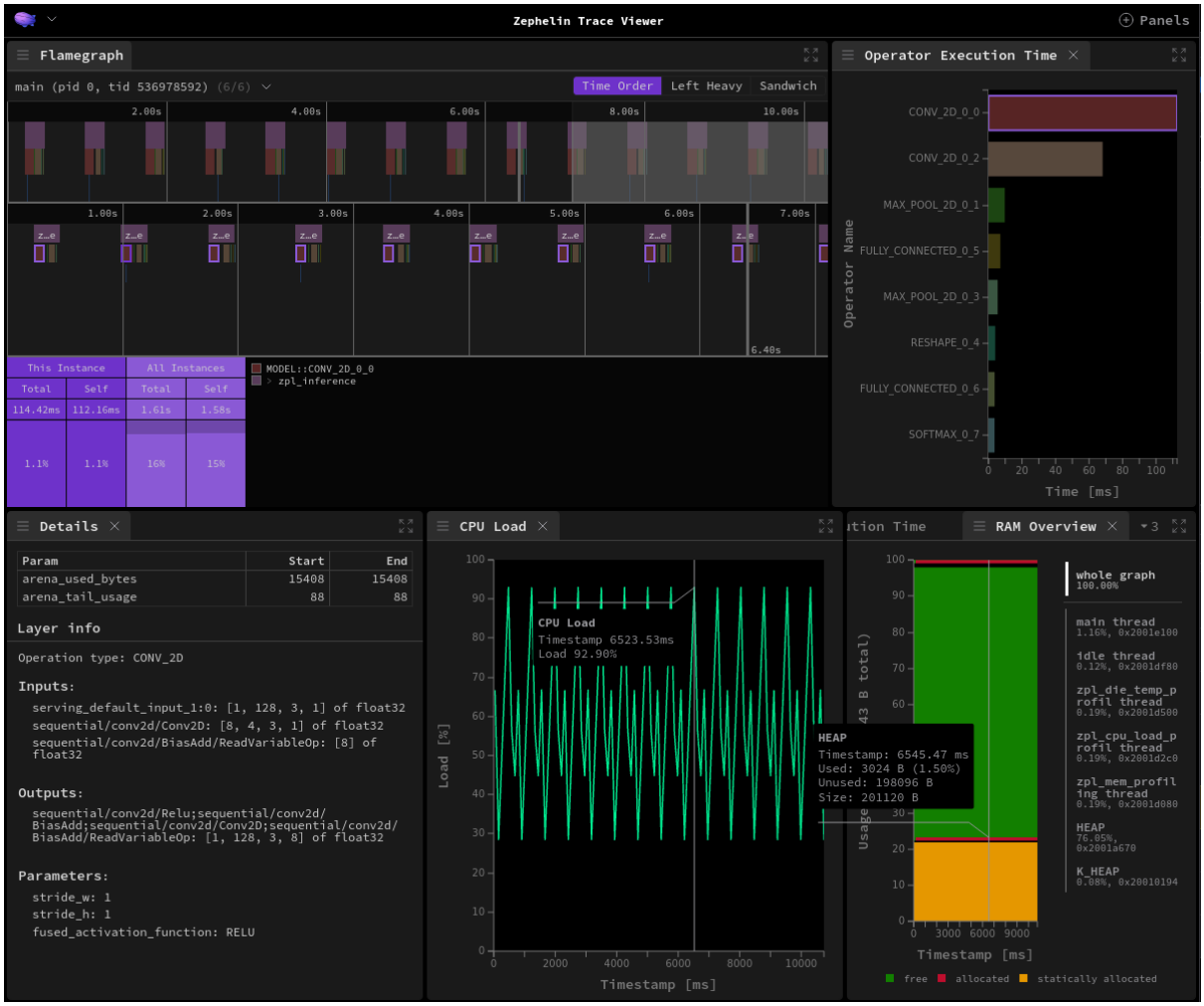


Fig. 11.4: The memory usage example

CHAPTER

TWELVE

LIVE TRACING OF EVENTS FROM THE BOARD

This chapter covers the usage of *Zephelin Trace Viewer* for live visualization of traces being gathered during application execution.

Real-time visualization requires running a local Python server responsible for ingesting CTF traces, parsing them into TEF, and forwarding them to the visualizer.

12.1 Real-time tracing server structure

Compared to the diagram from *Zephelin trace collection*, instead of saving and converting complete CTF traces to TEF and uploading them to Zephelin Trace Viewer, we continuously deliver traces over TCP to a Zephelin server, convert them chunk by chunk to TEF and deliver to Zephelin Trace Viewer over Remote Procedure Calls (RPC).

The Zephelin server consists of:

- Trace Handler - configures Parser Proxy, the `libbtrace` live plugin and the RPC Dispatcher and controls the delivery of traces and commands to the Zephelin Trace Viewer.
- RPC Dispatcher - sends and receives messages from Zephelin Trace Viewer.
- Stream Parser Proxy - collects data from `west zpl-<backend>-capture` over TCP and subdivides it to the commands' stream and traces. Traces are delivered to `libbtrace` live plugin.
- `libbtrace` live plugin - receives parts of CTF traces and converts them to parts of TEF messages that are later delivered to Trace Handler.

12.2 Prerequisites

Before running the server, you need to install required backend dependencies and compile the frontend visualizer.

12.2.1 Install server dependencies

Apart from installing Zephelin dependencies (see *Initializing the workspace*), server dependencies need to be installed as well with the following command:

```
pip install -r server/requirements.txt
```

12.2.2 Basic usage

To run the server with default settings and serve the frontend, call:

```
python server/run_backend.py
```

or `west` subcommand:

```
west zpl-live-server
```

12.2.3 Configuration Options

You can customize the server's networking interfaces, and specify trace parsing options that will be used in the *CTF to TEF* conversion.

- `--tcp-server-host` - Address of the Zephelin TCP socket for CTF trace ingestion (Default: `127.0.0.1`).
- `--tcp-server-port`- Port of the Zephelin TCP socket for CTF trace ingestion (Default: `5000`).
- `--backend-host` - Address where the backend API and visualizer are hosted (Default: `127.0.0.1`).
- `--backend-port` - Port where the backend API and visualizer are hosted (Default: `8000`).
- `--bt-port` - Port used by `libbtrace` live plugin to collect pure CTF traces (Default: `42674`).
- `--frontend-directory` - Path to compiled frontend directory (for downloading and building frontend check Zephelin Trace Viewer).
- `--build-dir` - Path to the traced application build directory.
- `--tflm-model-paths` - Paths to the TFLM models.
- `--tvm-model-paths` - Paths to the TVM models.
- `--tvm-model-metadata-paths` - Paths to the TVM model metadata files.
- `--tvm-model-op-remove-prefix` - Regex pattern used for removing TVM operator prefixes.
- `--tvm-model-op-remove-suffix` - Regex pattern used for removing TVM operator suffixes.
- `--verbosity` - Set the logging verbosity level (`DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`).

> **Note**
>
> If `--frontend-directory` is not provided with built website, the script downloads the latest built version from Zephelin Trace Viewer's GitHub.

12.2.4 Additional configuration options for tests

There is a possibility to run the server in mock mode, where TEF traces are delivered as inputs and sent to the website at specified speed:

- `--mock-trace-file` - Path to the TEF/JSON trace file to use for the mock.
- `--mock-playback-speed` - Playback speed multiplier for the mock.

12.3 Providing traces to the server

Once the server is running, it listens for CTF traces on the TCP socket. To route the traces read from one of the available backends (described in *Trace Collection* section), the `--send-to-remote` argument can be provided with the TCP socket address specified.

Example:

```
west zpl-uart-capture /dev/pts/12 115200 --send-to-remote 127.0.0.1:5000
```

12.4 Live-tracing controls

If the Visualizer is connected to the server, the live-tracing controls are available on the top-bar.

- `Start streaming` - enables continuous rendering of received traces.
- `Stop streaming` - disables continuous rendering.
- `Collect` - visualizes all the traces gathered so far.
- `Stop Tailing` - Stops the default behavior in which viewport follows the live edge of the visualized trace.
- `Resume Tailing` - Snaps the viewport back to the live edge.

> **Note**
>
> Additionally, `Alt+R` performs a hard reset of the tracing session, which restarts the `libbtrace` instance parsing the trace. It should only be used to recover from a `libbtrace` failure, as it drops the traces gathered so far.
>
> Such failure can manifest itself as the Trace Viewer no longer receiving any new events, even though the capture command keeps streaming data to the server, with a `libbtrace` error traceback printed in the server log.

12.5 Sample collection of traces

This sample demonstrates how to run an application running profiling for a TensorFlow Lite Micro model.

First, build a sample application running two TensorFlow Lite Micro models, with an increased number of iterations:

```
west build -p -b max32690fthr/max32690/m4 samples/profiling/tflm_multi_model -- -
↪DCONFIG_ZPL_TRACE_FORMAT_CTF=y -DCONFIG_TRACING_BUFFER_SIZE=10000 -DCONFIG_BOOT_
↪BANNER=n -DCONFIG_PRINTK=n -DCONFIG_LOG=n -DCONFIG_ZPL_SAMPLE_TFLM_NUM_ITERS=200
```

After this, run the server for live tracing, providing the paths to the models:

```
west zpl-live-server --tflm-model-paths ./samples/common/tflm/model/magic-wand.
↪tflite ./samples/common/tflm/model/sine.tflite
```

Then, run the collection of traces from:

- Renode:

```
python scripts/run_renode.py --trace-output test.ctf --send-to-remote 127.0.
↪0.1:5000
```

- From physical hardware (after flash), e.g.:

```
west zpl-uart-capture <path-to-uart> 115200 ./trace-hw.ctf --send-to-remote␣
↪127.0.0.1:5000
```

12.6 Live tracing with instrumentation

Traces produced by the *instrumentation subsystem* can be visualized live as well, as long as they are delivered over the same channel as regular Zephelin traces. This is the case for the tracing subsystem backend (`CONFIG_INSTRUMENTATION_BACKEND_TRACING_CORE=y`), which packs instrumentation events into a separate CTF stream of the regular trace. With that, no additional capture command is needed - a single `west zpl-<backend>-capture` with `--send-to-remote` feeds both streams to the server.

We will use the samples/profiling/tflm_instrumentation sample, which runs a TFLM model with both Zephelin profilers and the instrumentation subsystem enabled.

First, build the sample with the instrumentation tracing backend configuration:

```
west build -p -b max32690fthr/max32690/m4 samples/profiling/tflm_instrumentation -
↪- \
  -DEXTRA_CONF_FILE="zpl.conf;instrumentation_tracing.conf"
```

Then, start the server, pointing it to the model used by the sample and to the build directory:

```
west zpl-live-server \
  --tflm-model-paths ./samples/common/tflm/model/sine.tflite \
  --build-dir ./build
```

Next, run the collection of traces from:

- Renode:

```
python scripts/run_renode.py --send-to-remote 127.0.0.1:5000 --pause
```

- From physical hardware (after flash), e.g.:

```
west zpl-uart-capture <path-to-uart> 115200 ./trace-hw.ctf --send-to-remote␣
↪127.0.0.1:5000
```

Note

Starting the simulation with `--pause` allows to postpone the execution of the sample until the frontend is set up and ready. When using physical hardware, the server will only start processing traces after board reset.

Once the simulation is connected to the backend, open the visualizer at the configured address (default: `http://127.0.0.1:8000`), press `Start streaming` and unpause the simulation.

12.7 Communication flow

The server implementation in `server/run_backend.py` (also available as `west zpl-live-server`) consists of the following access points:

- `<tcp_server_host>:<tcp_server_port>` - allows for communication between `west` capture subcommands and the server.
- `<backend_host>:<backend_port>` - hosts the website loaded from the `--frontend-directory` directory and configures the communication between website and server.
- `127.0.0.1:<bt_port>` - allows for delivery of pure CTF events to the `libbtrace` live plugin (it is a one-way communication).

Like in the file-based Zephelin flow, the traces are first collected and sent by the board to the host using a selected backend (e.g. UART). After this, the `west zpl-<backend>-capture` command with the `--send-to-remote` flag configured to e.g. `127.0.0.1:5000` will send CTF traces with additional data (such as `_zpl_ctf_start__` start tag) to a given address.

This address should match the `--tcp-server-host` and `--tcp-server-port` settings in the `server/run_backend.py` script. `Stream Parser Proxy` will receive packets sent by the `west` subcommand and subdivide them into commands (such as restart the viewer based on start tag) that should be sent as RPC commands to the website, and pure CTF events.

Pure CTF events are sent to the `libbtrace` live plugin over the `--bt-port` port on loopback (e.g. `127.0.0.1:42674`), for fast conversion to TEF events in C++. The plugin communicates with the `libbtrace` module running in the server, delivering `bt2` messages with parts of parsed TEF events - it is not socket-based communication.

> **Note**
>
> It is possible to deliver pure CTF events directly to the `libbtrace` live plugin by sending events directly to `--bt-port`, skipping the `Stream Parser Proxy`.

Those `bt2` messages are delivered to the `Trace Handler`. Trace Handler performs final adjustments of the TEF events and sends them to the website.

In the meantime, `RPC Dispatcher` allows for sending calls between server and website, and controls the state of Zephelin Trace Viewer using procedures described in *Server API reference*.

12.8 Server API reference

The Zephelin real-time tracing server supports the following JSON-RPC requests sent as rpc_request WebSocket event:

- `trace.connect`

  async handlers.trace_handler.TraceHandler.**connect**(self) → dict['status' | 'message', str]

  Starts the TCP server to listen for incoming trace streams from capture scripts.

12.8.1 Returns

dict[Literal[“status”, “message”], str]
Status message.

- `trace.disconnect`

async handlers.trace_handler.TraceHandler.**disconnect**(self) → dict['status' | 'message', str]

Terminates the background read task and tears down transportation backend.

12.8.2 Returns

dict[Literal[“status”, “message”], str]
Status message.

- `trace.stream_start`

async handlers.trace_handler.TraceHandler.**stream_start**(self) → dict['status', str]

Enables continuous trace streaming to the frontend.

12.8.3 Returns

dict[Literal[“status”], str]
Status message.

- `trace.stream_stop`

async handlers.trace_handler.TraceHandler.**stream_stop**(self) → dict['status', str]

Disables continuous trace streaming to the frontend.

12.8.4 Returns

dict[Literal[“status”], str]
Status message.

- `trace.metadata`

async handlers.trace_handler.TraceHandler.**metadata**(self) → dict['status' | 'message' | 'data', str | dict]

Provides model metadata and memory symbols for the trace.

Runs the heavy collection work in a background thread so the async event loop (and thus the streaming pipeline) is not blocked. Once the work finishes any events that accumulated in the emit queue during the collection are flushed immediately.

12.8.5 Returns

dict[Literal[“status”, “message”, “data”], Union[str, dict]]
Message with metadata events or error.

- `trace.reset`

async handlers.trace_handler.TraceHandler.**reset**(self)

Resets the trace buffer.

- trace.collect

async handlers.trace_handler.TraceHandler.**collect**(self) → dict['status' | 'message' | 'data', str | dict]

Provides the increment of the trace buffer not yet sent.

12.8.6 Returns

dict[Literal[“status”, “message”, “data”], Union[str, dict]]

Message with all events present in the buffer or error.

CHAPTER

THIRTEEN

ZEPHELIN USAGE EXAMPLES

This chapter provides links to relevant resources, application examples and *Trace Viewer* views.

Note

For details on how each demo is executed, open the collapsible description of the launch example.

Before running commands, make sure you went through *Initializing the workspace* and *Running sample project in Zephelin* as it is needed to prepare the environment.

13.1 Using code scopes

- **Source**: samples/basic/marking_code_scopes
- **Trace Viewer**: preview

This sample demonstrates the usage of `ZPL_MARK_CODE_SCOPE` which was described in *Code scopes chapter*.

Generating traces for code with scopes defined

To build a sample run:

```
west build -p -b max32690fthr/max32690/m4 samples/basic/marking_code_scopes --␣
↪-DCONFIG_ZPL_TRACE_FORMAT_CTF=y -DCONFIG_TRACING_BUFFER_SIZE=10000 -DCONFIG_
↪BOOT_BANNER=n -DCONFIG_PRINTK=n -DCONFIG_LOG=n
```

The traces can be either obtained from hardware or a Renode simulation with:

Renode

Hardware

For Renode, the simulation and collection of traces can be done with:

```
python3 ./scripts/run_renode.py --trace-output ./trace.ctf --timeout 45
```

For hardware, once the device is flashed, the traces can be collected with:

```
west zpl-uart-capture /dev/ttyUSB0 115200 ./trace.ctf
```

Then, the TEF traces can be created with `west zpl-prepare-trace` like so:

```
west zpl-prepare-trace ./trace.ctf -o ./tef_marking_code_scopes.json
```

In the end, the generated `tef_marking_code_scopes.json` can be loaded in Trace Viewer.

13.2 Simple TFLite Micro profiling

- **Source**: samples/profiling/tflm_profiler
- **Trace Viewer**: preview

This sample demonstrates tracing of the TensorFlow Lite Micro (or LiteRT) runtime. The collection of traces is performed as usual, but in `west zpl-prepare-trace` you need to provide the `--tflm-model-path <path-to-model>` flag with the used model.

Generating traces for TFLite Micro runtime

To build a sample, run:

```
west build -p -b max32690fthr/max32690/m4 samples/profiling/tflm_profiler -- -
↪DCONFIG_ZPL_TRACE_FORMAT_CTF=y -DCONFIG_TRACING_BUFFER_SIZE=10000 -DCONFIG_
↪BOOT_BANNER=n -DCONFIG_PRINTK=n -DCONFIG_LOG=n
```

The traces can be either obtained from hardware or a Renode simulation with:

Renode

Hardware

For Renode, the simulation and collection of traces can be done with:

```
python3 ./scripts/run_renode.py --trace-output ./trace.ctf --timeout 45
```

For hardware, once the device is flashed, the traces can be collected with:

```
west zpl-uart-capture /dev/ttyUSB0 115200 ./trace.ctf
```

Then, the TEF traces can be created with `west zpl-prepare-trace` like so:

```
west zpl-prepare-trace ./trace.ctf \
  --tflm-model-path ./samples/common/tflm/model/magic-wand.tflite \
  -o ./tef_tflm_profiler.json
```

In the end, the generated `tef_tflm_profiler.json` can be loaded in Trace Viewer.

13.3 Simple microTVM profiling

- **Source**: samples/profiling/tvm_profiler
- **Trace Viewer**: preview

This sample demonstrates tracing of the microTVM runtime. The collection of traces is performed as usual, but in `west zpl-prepare-trace` you need to provide:

- `--tvm-model-path <path-to-model-graph>` - path to JSON with model graph
- `--tvm-model-metadata-path <path-to-metadata>` - path to metadata from compilation results

For more details on the above flags, check *CTF to TEF conversion*.

Generating traces for microTVM runtime

To build a sample run:

```
west build -p -b max32690fthr/max32690/m4 samples/profiling/tvm_profiler -- -
↪DCONFIG_ZPL_TRACE_FORMAT_CTF=y -DCONFIG_TRACING_BUFFER_SIZE=10000 -DCONFIG_
↪BOOT_BANNER=n -DCONFIG_PRINTK=n -DCONFIG_LOG=n
```

The traces can be either obtained from hardware or a Renode simulation with:

Renode

Hardware

For Renode, the simulation and collection of traces can be done with:

```
python3 ./scripts/run_renode.py --trace-output ./trace.ctf --timeout 45
```

For hardware, once the device is flashed, the traces can be collected with:

```
west zpl-uart-capture /dev/ttyUSB0 115200 ./trace.ctf
```

Then, the TEF traces can be created with `west zpl-prepare-trace` like so:

```
west zpl-prepare-trace ./trace.ctf \
  --tvm-model-path ./samples/common/tvm/model/magic-wand-graph.json \
  --tvm-model-metadata-path ./samples/common/tvm/model/magic-wand-metadata.
↪json \
  -o ./tef_tvm_profiler.json
```

In the end, the generated `tef_tvm_profiler.json` can be loaded in Trace Viewer.

13.4 Full TFLite Micro traces with instrumentation using separate UART connection for instrumentation traces

- **Source**: samples/profiling/tflm_instrumentation
- **Trace Viewer**: preview

This sample demonstrates combining Zephelin tracing with enabled instrumentation and additional metrics.

The collection of traces is performed using `west zpl-instrumentation-uart-gdb-capture` which collects:

- Instrumentation data using UART
- Zephelin data using GDB backend

In this scenario, you need to provide the instrumentation traces separately using the `-i` flag.

Check *CTF to TEF conversion* for more details.

Collecting regular and instrumentation traces

To build a sample run:

```
west build -p -b max32690fthr/max32690/m4 samples/profiling/tflm_
↪instrumentation -- -DEXTRA_CONF_FILE="instrumentation_uart.conf;zpl.conf"
```

Secondly, flash the device or run a simulation and collect traces:

Renode

Hardware

For Renode, the simulation and collection of traces can be done with:

```
python3 ./scripts/run_renode.py --simulation-only --debug
```

This can run in a separate shell, since traces in this case will be obtained separately with the following command:

```
west zpl-instrumentation-uart-gdb-capture /tmp/uart-log 115200 ./renode_tflm.
↪instr.ctf ./renode_tflm.gdb.ctf --no-debug-server --timeout 20
```

For hardware, flash the device, and make sure that UART and a debug adapter is connected.

Once this is done, run the collection of traces with:

```
west zpl-instrumentation-uart-gdb-capture /dev/ttyUSB0 115200 ./renode_tflm.
↪instr.ctf ./renode_tflm.gdb.ctf --timeout 20
```

Remember that `/dev/ttyUSB0` may need to be replaced with a different device.

Then, the TEF traces can be created with `west zpl-prepare-trace` like so:

```
west zpl-prepare-trace -o tef_tflm_instrumentation.json -i renode_tflm.instr.
↪ctf renode_tflm.gdb.ctf --tflm-model-path ./samples/common/tflm/model/sine.
↪tflite --trim-metadata
```

In the end, the generated `tef_tflm_instrumentation.json` can be loaded in Trace Viewer.

Note

The `renode_tflm.instr.ctf` can be saved under a different path (e.g. `renode_tflm.instr.ctf`) due to resets - check paths reported by `west zpl-instrumentation-uart-gdb-capture` and update the command accordingly.

13.5 Full TFLite Micro traces with instrumentation sent through tracing subsystem

- **Source**: samples/profiling/tflm_instrumentation
- **Trace Viewer**: preview

This sample demonstrates how to collect regular traces, custom traces, execution metrics, model-specific metrics and instrumentation traces using only the tracing subsystem transport, without delivering instrumentation traces separately.

In this scenario, instead of `west zpl-instrumentation-uart-gdb-capture`, you can use regular Zephelin capture commands, such as `west zpl-uart-capture`.

Collecting regular and instrumentation traces using `zpl-uart-capture`

To build a sample, run:

```
west build -p -b max32690fthr/max32690/m4 samples/profiling/tflm_
↪instrumentation -- -DEXTRA_CONF_FILE="instrumentation_tracing.conf;zpl.conf"
```

Secondly, flash the device or run a simulation and collect traces:

Renode

Hardware

For Renode, the simulation and collection of traces can be done with:

```
python3 ./scripts/run_renode.py --trace-output ./trace.ctf --timeout 45
```

For hardware, once the device is flashed, the traces can be collected with:

```
west zpl-uart-capture /dev/ttyUSB0 115200 ./trace.ctf
```

Then, the TEF traces can be created with `west zpl-prepare-trace` like so:

```
west zpl-prepare-trace ./trace.ctf \
  --tflm-model-path ./samples/common/tflm/model/sine.tflite \
  -o ./tef_tflm_instrumentation_tracing.json
```

In the end, the generated `./tef_tflm_instrumentation_tracing.json` can be loaded in Trace Viewer.

13.6 Multithreaded application running multiple models

- **Source**: samples/profiling/smp_tvm
- **Trace Viewer**: preview (models are running on a different thread than `main`)

Zephelin allows for tracing of applications running on multiple threads and cores with no additional configuration changes. Each event contains the ID of the CPU which emitted it; this ID is used during parsing traces.

This sample loads four simple models using TVM runtime. Those models are then executed on different CPUs using random input data generated by the main thread.

Tracing multiple threads with Zephelin

To build a sample, run:

```
west build -p -b  mpfs_icicle/polarfire/u54/smp samples/profiling/smp_tvm
```

Note

The `mpfs_icicle/polarfire/u54/smp` is used to demonstrate the work on several cores.

The above sample can be executed on hardware or simulated in Renode with:

```
python ./scripts/run_renode.py --trace-output ./smp.ctf
```

Finally, to parse produced `./smp.ctf` run:

```
west zpl-prepare-trace ./smp.ctf -o ./tef_smp_tvm_models.json \
  --tvm-model-paths ./samples/common/tvm/model/sine-graph.json \
    ./samples/common/tvm/model/magic-wand-graph.json \
  --tvm-model-metadata-paths ./samples/common/tvm/model/sine-metadata.json \
    ./samples/common/tvm/model/magic-wand-metadata.json \
  --tvm-model-op-remove-prefix 'tvmgen_[a-zA-Z0-9]+_fused_' \
  --trim-metadata
```

In the Trace Viewer, there should be a separate thread for each model; each of those threads should contain events produced by that model.

13.7 Simple tracing of two applications running on different SoCs with common external clock

- **Source**: samples/multi_machine/two_models
- **Trace Viewer**: preview

This sample demonstrates the tracing of two Zephyr applications running on separate boards.

Those boards share a common clock provider, allowing for traces to be synchronized. An artificial time offset is introduced for demonstration purposes.

Tracing the same solution on two machines

To build a sample, run:

```
west build -p -b max32650fthr --sysbuild samples/multi_machine/two_models/node_
↪0 -- \
   -Dnode_0_CONFIG_ZPL_TRACE_FORMAT_CTF=y \
   -Dnode_0_CONFIG_TRACING_BUFFER_SIZE=10000 \
   -Dnode_1_CONFIG_ZPL_TRACE_FORMAT_CTF=y \
   -Dnode_1_CONFIG_TRACING_BUFFER_SIZE=10000
```

The demonstration uses the same application for both boards.

The above sample can be simulated in Renode with:

```
python3 ./scripts/run_renode_multimachine.py \
        --boards max32650fthr max32650fthr \
        --elfs build/node_0/zephyr/zephyr.elf build/node_1/zephyr/zephyr.elf \
        --trace_uarts uart0 uart0 \
        --shared_clock_address 0x400FFFF0 \
        --offset 2000 \
        --trace-output ./trace.ctf \
        --timeout 20
```

To parse the produced `./trace.ctf` and `./trace_1.ctf`, run:

```
west zpl-prepare-trace ./trace.ctf \
  --build-dir build/node_0 \
  --tflm-model-path ./samples/common/tflm/model/magic-wand.tflite \
  --zephyr-elf-path build/node_0/zephyr/zephyr.elf \
  -o ./tef_tflm_profiler_0.json
west zpl-prepare-trace ./trace_1.ctf \
  --build-dir build/node_1 \
  --tflm-model-path ./samples/common/tflm/model/magic-wand.tflite \
  --zephyr-elf-path build/node_1/zephyr/zephyr.elf \
  -o ./tef_tflm_profiler_1.json
```

In Trace Viewer, traces from the same model run should be available but with 2s offset between them.

13.8 Preprocessing to micro-speech pipeline

- **Source**: samples/multi_machine/micro_speech
- **Trace Viewer**: preview

This sample demonstrates deployment of a Micro-speech model on two SoCs connected via UART. The preprocessor running on the first SoC converts a raw audio sample into a spectrographic feature. The processed data is passed via UART to the second SoC, which runs a Micro-speech model once sufficient data has been accumulated. The model responses can be `yes`, `no`, `silence` and `undefined`. The sample expects `.h` files as input. To prepare the `.wav` file, you can use following *script*. Example usage:

```
python3 wav_to_h.py input.wav output.h
```

Micro-speech pipeline

To build a sample, run:

```
west build -p -b max32650fthr --sysbuild samples/multi_machine/micro_speech/
↪micro_speech -- \
   -Dmicro_speech_CONFIG_ZPL_TRACE_FORMAT_CTF=y \
   -Dmicro_speech_CONFIG_TRACING_BUFFER_SIZE=10000 \
   -Dpreprocessor_CONFIG_ZPL_TRACE_FORMAT_CTF=y \
   -Dpreprocessor_CONFIG_TRACING_BUFFER_SIZE=10000
```

The above sample can be executed on hardware or simulated in Renode with:

```
python3 ./scripts/run_renode_multimachine.py \
--boards max32650fthr max32650fthr \
--elfs build/micro_speech/zephyr/zephyr.elf build/preprocessor/zephyr/zephyr.
↪elf \
--repls samples/multi_machine/boards/max32650fthr.repl samples/multi_machine/
↪boards/max32650fthr.repl \
--trace_uarts uart0 uart0 \
--uart-connect uart1 \
--shared-clock-address 0x400FFFF0 \
--trace-output micro_speech.ctf \
--timeout 45
```

To parse the produced `./microspeech.ctf` and `./microspeech_1.ctf`, run:

```
west zpl-prepare-trace ./micro_speech.ctf \
  --build-dir build/micro_speech \
  --tflm-model-path ./samples/common/tflm/model/micro_speech_quantized.tflite \
  -o ./tef_micro_speech.json \
  --zephyr-elf-path build/micro_speech/zephyr/zephyr.elf

west zpl-prepare-trace ./micro_speech_1.ctf \
  --build-dir build/micro_speech \
  --tflm-model-path ./samples/common/tflm/model/audio_preprocessor_int8.tflite␣
↪\
  -o ./tef_preprocessor.json \
  --zephyr-elf-path build/preprocessor/zephyr/zephyr.elf
```

CHAPTER

FOURTEEN

TROUBLESHOOTING

This chapter collects the most common problems encountered when building applications with Zephelin, capturing traces and converting them for Zephelin Trace Viewer, together with their causes and solutions.

14.1 Trace capture

14.1.1 The capture command produces an empty or truncated file

Check the following, in order:

- `CONFIG_ZPL_TRACE` is enabled and one of the trace formats (`CONFIG_ZPL_TRACE_FORMAT_CTF` or `CONFIG_ZPL_TRACE_FORMAT_PLAINTEXT`) is selected.
- The backend enabled in Kconfig matches the `west zpl-*-capture` command used - firmware built with `CONFIG_ZPL_TRACE_BACKEND_DEBUGGER` will not produce traces on UART.
- The capture is running while the traced code executes.

14.1.2 The captured data is incomprehensive

- Verify you are reading the trace UART, not the console UART. The samples select the trace UART with the `zephyr,tracing-uart` chosen property in a board overlay (see samples/common/boards). If application logs are interleaved with the CTF stream, the trace is unparsable.
- With `CONFIG_ZPL_TRACE_FORMAT_CTF`, the file is binary by design - convert it with `west zpl-prepare-trace` into a `.tef` file and load it into Trace Viewer for inspection. Traces generated with `CONFIG_ZPL_TRACE_FORMAT_PLAINTEXT` cannot be converted.

14.1.3 Events are missing or seem to never end

Events are staged in the Zephyr tracing buffer sized with `CONFIG_TRACING_BUFFER_SIZE` before the backend sends them out. How an overflow manifests depends on the tracing mode:

- With `CONFIG_TRACING_ASYNC` (the Zephyr default), the buffer is a ring buffer and the packets that do not fit are dropped, which shows up as missing or unending events rather than as an error. Increase `CONFIG_TRACING_BUFFER_SIZE` - most samples use `4096`, and samples/profiling/smp_tvm needs `10240` - and, with the UART backend, use the highest baud rate supported by the board.
- With `CONFIG_TRACING_SYNC`, which most Zephelin samples enable, the buffer only holds a single formatted packet, so it has to be at least `CONFIG_TRACING_PACKET_MAX_SIZE` bytes.

Nothing is dropped in this mode, but every event blocks the emitting thread until the backend accepts it.

14.1.4 USB capture does not find the device

`west zpl-usb-capture` fails with `Couldn't open USB device with vid=... pid=...` when the device is not enumerated yet or the identifiers do not match.

- Use `-w/--wait-for-device` to wait for the board instead of failing immediately, which is useful when the board is reset after the command starts.
- Make sure the USB requirements described in *Trace capture USB interface requirements* are met.

14.2 Converting traces

14.2.1 `Zephyr CTF metadata (...) does not exist`

The conversion merges the Zephyr and Zephelin CTF metadata, and the Zephyr part is looked up relative to the Zephyr repository. By default, the path is deduced from the script location inside the workspace, which fails when Zephelin is used outside of it.

Pass `--zephyr-base` to `west zpl-prepare-trace` or export `ZEPHYR_BASE`.

14.2.2 Function names in the trace are mangled

C++ symbols are demangled with `c++filt` taken from the Zephyr SDK. If the conversion prints `c++filt is missing`, point Zephelin at a working binary with the `ZPL_DEMANGLE_CMD` environment variable.

14.2.3 Instrumentation events show raw addresses instead of function names

`No symbols found - using callee address ...` means the symbols could not be resolved from the ELF file. The instrumentation trace has to be converted against the very same binary that was flashed on the device, so pass the matching `--build-dir` or `--zephyr-elf-path` and re-flash the board if the build directory has been rebuilt in the meantime.

14.3 Live tracing

Live tracing is driven by the Zephelin server, which can be started either with `python server/run_backend.py` or with the `west zpl-live-server` command. Both accept the same arguments and can be used interchangeably, so the options mentioned below apply to either of them.

14.3.1 The viewer stays empty although the capture command reports incoming bytes

The server discards all incoming data until it finds the `_zpl_ctf_start__` synchronization tag, which is emitted by the device on startup. If the capture is started while the application is already running, the tag never arrives.

Reset the board after connecting the capture, or start the Renode simulation with `--pause` and unpause it once the frontend is connected, as described in *Live tracing of events from the board*.

14.3.2 CPU load, memory or die temperature samples only appear at the end of the trace

The periodic profilers run in their own threads, with a default priority of 5. An application that runs inferences back-to-back in a higher-priority thread without yielding starves them, so their events are only emitted once the busy loop ends, even though the trace pipeline works correctly.

Either add sleeps to the application loop, or raise the priority of the profiling threads, for example:

```
CONFIG_ZPL_CPU_LOAD_PROFILING_THREAD_PRIORITY=-1
CONFIG_ZPL_MEMORY_PROFILING_THREAD_PRIORITY=-1
```

The profiling threads sleep between the samples, so making them cooperative does not block the application.

14.3.3 Traces do not reach the server at all

`--send-to-remote` has to point at the TCP socket used for CTF ingestion, that is `--tcp-server-port` (5000 by default):

```
west zpl-uart-capture /dev/ttyUSB0 115200 ./trace.ctf --send-to-remote 127.0.0.
↪1:5000
```

Sending the data to `--bt-port` (42674 by default) instead bypasses the trace handler - this port is used internally between the server and the `libbtrace` live plugin.

14.3.4 Layer profiling events are missing or appear to last forever in live mode only

Delayed emission makes the trace stream non-chronological, and the live pipeline renders the events as they arrive, dropping the ones older than the newest event already displayed for a given thread. Offline conversion sorts the whole trace, so the same run visualized from a file is complete.

Disable `CONFIG_ZPL_TFLM_PROFILER_DELAYED_EMISSION` or `CONFIG_ZPL_TVM_PROFILER_DELAYED_EMISSION` when using live tracing.

CHAPTER

FIFTEEN

ZEPHELIN'S DEVELOPER GUIDELINES

This chapter covers and links to resources and paths related to development and enhancing Zephelin library.

15.1 General Zephyr development guidelines

With regard to code styling and configuration, this project follows Zephyr's development guidelines and linting rules - for more details, check General guidelines for Zephyr.

For setting up the development environment, check the *Zephelin project introduction*.

15.2 Adding build-time and runtime configurations for Zephelin

Instructions on how to introduce new configuration options, available either during build time or when the application is running, can be found in *Adding new configurations*.

15.3 Adding support for new AI inference libraries

Details on how to introduce support for new AI runtimes is explained in detail in a separate chapter - *Adding support for new AI inference libraries*.

15.4 Adding support for new trace backends

Existing implementations of backends can be found in zpl/backends and in zephyr/subsys/tracing

To allow for sending trace data through a new backend, assuming a relatively simple case, you need to implement two methods:

- A method for initializing the backend, e.g.:

```
static void tracing_backend_custom_impl_init(void);
```

- A method for yielding trace data to the implemented communication method, e.g.:

```
static void tracing_backend_custom_impl_output(const struct tracing_backend␣
↪*backend, uint8_t *data, uint32_t length);
```

Then, prepare a `tracing_backend_api` struct with the above methods (see zephyr/subsys/tracing/include/tracing_backend.h for more details):

```
const struct tracing_backend_api tracing_backend_custom_impl_api = {
    .init = tracing_backend_custom_impl_init,
    .output = tracing_backend_custom_impl_output
};
```

Register the backend using `TRACING_BACKEND_DEFINE` from zephyr/subsys/tracing/include/tracing_backend.h:

```
TRACING_BACKEND_DEFINE(tracing_backend_custom_impl, tracing_backend_custom_impl_
↪api);
```

You can define a corresponding entry in zpl/backends/Kconfig, e.g. `CONFIG_ZPL_TRACE_BACKEND_UART`, and configure the build for the backend in zpl/backends/CMakeLists.txt:

```
zephyr_library_sources_ifdef(CONFIG_ZPL_TRACE_BACKEND_UART custom_impl_backend.c)
```

The Renode Trivial UART backend (zpl/backends/trivial_uart_backend.c) can be used as a simple example of adding new backends to Zephelin.

15.5 Adding support for tracing new data sources in Zephelin

Existing implementations of tracing specific data sources (CPU load, temperature, ...) can be found in zpl/profilers.

The implementation of tracing custom data in the application boils down to:

- Defining a new event type
- Defining a function for reading data and creating an event and printing it
- (depending on source type) Setting up a separate thread to collect traces

We will discuss how this can be implemented on an example of CPU load.

First, you need to define an event that will store the collected sample - a packed `struct` can be defined in a dedicated header in include/zpl:

```
#ifdef __cplusplus
extern "C" {
#endif

#if defined(CONFIG_ZPL_TRACE_FORMAT_CTF)
/* CPU load event ID */
#define ZPL_CPU_LOAD_EVENT 0xC0

/**
 * CPU load event structure.
 */
typedef struct __packed {
        uint16_t stream_id;
        uint16_t packet_size;
        uint64_t timestamp;
```

(continues on next page)

(continued from previous page)

```
        uint16_t id;
        uint8_t cpu_id;
        uint16_t cpu_load; /* CPU load denoted by a number 0-1000 */
} zpl_cpu_load_event_t;
#endif /* defined(CONFIG_ZPL_TRACE_FORMAT_CTF) */

/**
 * Emits CPU load event.
 */
void zpl_emit_cpu_load_event(void);

#ifdef __cplusplus
}
#endif
```

The `ZPL_CPU_LOAD_EVENT 0xC0` is a unique event ID - for the list of existing events' IDs, check either the zpl/metadata event metadata file or in *Events documentation*, which describe events for both CTF and TEF. The `zpl_cpu_load_event_t` structure holds:

- `stream_id` - ID of the stream for data; currently `1` is used for AI runtimes and `0` for the remaining events
- `cpu_id` - ID of the CPU on which the event is collected
- `id` - ID of the event, in here it should be `ZPL_CPU_LOAD_EVENT`
- `timestamp` - timestamp for the event, in nanoseconds
- `packet_size` - size of the entire structure in bits
- event-specific data

The corresponding event entry needs to be created in zpl/metadata:

```
event {
        stream_id = 0;
        name = zpl_cpu_load_event;
        id = 0xC0;
        fields := struct {
                uint8_t cpu_id;
                uint16_t cpu_load;
        };
};
```

Later, you need to implement a function for collecting the traces - it needs to support either CTF or plain text format, and needs to be configurable at compile time or during execution. Implementation may look like so:

```
void zpl_emit_cpu_load_event(void)
{
        // do nothing if cpu_load is disabled
        ZPL_CONF_RETURN_IF_DISABLED(cpu_load);
        int cpu_load = cpu_load_get(true);
```

(continues on next page)

(continued from previous page)

```
#if defined(CONFIG_ZPL_TRACE_FORMAT_CTF)
        // Yielding CPU load data in CTF format
        // Acquire lock to avoid parallel emitting
        int key = irq_lock();
        // Define and fill the event
        zpl_cpu_load_event_t zpl_cpu_load_event = {
                .timestamp = zpl_timestamp_get(),
                .id = ZPL_CPU_LOAD_EVENT,
                .cpu_id = arch_curr_cpu()->id,
                .cpu_load = cpu_load,
                .stream_id = 0,
                .packet_size = sizeof(zpl_cpu_load_event_t) * 8,
        };
        // Emit the event
        tracing_format_raw_data(
                (uint8_t *)&zpl_cpu_load_event, sizeof(zpl_cpu_load_event_t)
        );
        irq_unlock(key);
#elif defined(CONFIG_ZPL_TRACE_FORMAT_PLAINTEXT)
        // Yielding CPU load data in plain text format
        TRACING_STRING("zpl_cpu_load_event: cpu_load=%d.%d%\n", cpu_load / 10,␣
↪cpu_load % 10);
#endif /* CONFIG_ZPL_TRACE_FORMAT_* */
}
```

The steps for the code are described in the comments. `ZPL_CONF_RETURN_IF_DISABLED` is described in *Zephelin configuration chapter*.

Lastly, you can implement the thread that will periodically read the CPU load and emit the event.

```
static void zpl_profile_cpu_load(void)
{
        while (true) {
                ZPL_CONF_WAIT(cpu_load_profiler);
                zpl_emit_cpu_load_event();
                k_msleep(CONFIG_ZPL_CPU_LOAD_PROFILING_INTERVAL);
        }
}

K_THREAD_DEFINE(zpl_cpu_load_profiling,
                CONFIG_ZPL_CPU_LOAD_PROFILING_THREAD_STACK_SIZE,
                zpl_profile_cpu_load, NULL, NULL, NULL,
                CONFIG_ZPL_CPU_LOAD_PROFILING_THREAD_PRIORITY,
                0, CONFIG_ZPL_CPU_LOAD_PROFILING_THREAD_DELAY);
```

This will emit events either in text or CTF. The CTF traces will later be parsed by the scripts/ctf2tef.py script in order to generate TEF traces for Trace Viewer to obtain - check *CTF to TEF conversion* for more details.

15.6 Adding Zephyr patches

To simplify the development of Zephyr patches, you can use `apply_patches.py` and `update_patches.py` scripts for applying patches and converting commits to patches.

Both scripts work on the patch set of one Zephyr version, and they take which one from `$ZPL_PATCH_BASE` rather than guessing, so source `./scripts/zephyr_version.sh` script first. By default it checks out `v4.3.0` release - to work on a different version, run `./scripts/switch_zephyr_version.sh <version>` first.

Before changing patches, ensure that the repositories are updated:

```
source ./scripts/zephyr_version.sh
west update
```

Then apply Zephyr patches using:

```
python scripts/apply_patches.py
```

In case new commits are introduced or existing commits are modified, the changes can be saved as patches with:

```
python scripts/update_patches.py
```

The resolver takes the version from the workspace, so these always act on the patch set that matches what is checked out. To work on a different version, run `./scripts/switch_zephyr_version.sh <version>` first.

15.7 Supporting a new Zephyr version

Each supported Zephyr version is described by four things, all named after the version:

- `zephyr-versions/<version>.env` - SDK URL, Python version and Renode package for CI
- `manifests/west-<version>.yml` - the West manifest
- `zephyr/patches/<version>/` - the patches
- `zephyr/patches/<version>.yml` - the patch manifest, generated by `update_patches.py`

To add a version, copy the closest existing set, adjust `revision:` and the `.env` values, then rebase the patches:

```
./scripts/switch_zephyr_version.sh <new-version>   # stops on the first␣
↪conflicting patch
source ./scripts/zephyr_version.sh
python scripts/apply_patches.py                    # re-apply as commits, up to␣
↪the conflict
# resolve the conflicts as commits in the module checkouts, then:
python scripts/update_patches.py
```

CHAPTER

SIXTEEN

ZEPHELIN BENCHMARKING

This chapter demonstrates the overhead introduced by the Zephelin library, based on its different variants.

16.1 Zephelin Benchmark Results

16.1.1 max32690fthr_max32690_m4

Benchmark	Cycles	FLASH [B]	RAM [B]	Configs
bench.basic (basic)	7583	4784	18032	
bench.basic.instr (basic)	21205802	21496	58780	*instr_tracing.conf*
bench.basic.tracing (basic)	7583	5048	20624	*tracing.conf*
bench.basic.zpl (basic)	237038	5080	21576	*zpl.conf*
bench.dhry (dhrystone)	2025	15176	20412	
bench.dhry.instr (dhrystone)	4695733	31880	61420	*instr_tracing.conf*
bench.dhry.tracing (dhrystone)	2027	15440	22984	*tracing.conf*
bench.dhry.zpl (dhrystone)	2027	15456	23592	*zpl.conf*
bench.tflite (tflite)	76433	29720	79276	*tflite.conf*
bench.tflite.tracing (tflite)	76052	29984	82772	*tflite.conf*, *tracing.conf*
bench.tflite.zpl (tflite)	84884	30040	83964	*tflite.conf*, *zpl.conf*
bench.threads (threads)	5950517	16216	19672	
bench.threads.tracing (threads)	40655082	17128	24252	*tracing.conf*

16.1.2 stm32f746g_disco_stm32f746xx

Benchmark	Cycles	FLASH [B]	RAM [B]	Configs
bench.basic (basic)	7951	4608	18552	
bench.basic.instr (basic)	29506628	21312	65848	*instr_tracing.conf*
bench.basic.tracing (basic)	7951	4800	21164	*tracing.conf*
bench.basic.zpl (basic)	183986	4864	21916	*zpl.conf*
bench.dhry (dhrystone)	2125	14976	20936	
bench.dhry.instr (dhrystone)	6528969	31744	68484	*instr_tracing.conf*
bench.dhry.tracing (dhrystone)	2125	15232	23528	*tracing.conf*
bench.dhry.zpl (dhrystone)	2125	15232	23932	*zpl.conf*
bench.tflite (tflite)	78292	29568	79804	*tflite.conf*
bench.tflite.tracing (tflite)	78029	29760	83332	*tflite.conf*, *tracing.conf*
bench.tflite.zpl (tflite)	84324	29824	84304	*tflite.conf*, *zpl.conf*
bench.threads (threads)	6576133	16832	20244	
bench.threads.tracing (threads)	47878785	17536	24856	*tracing.conf*

16.2 Benchmarks

16.2.1 ZPL_BENCHMARK_RUN_BASIC

Basic Zephelin benchmark (a very inefficient dot product implementaton)

Based on an inefficient dot product implementation. Uses ZPL scopes for tracing.

16.2.2 ZPL_BENCHMARK_RUN_DHRYSTONE

Dhrystone Zephelin benchmark

Synthetic benchmark for integer operations. Doesn't rely on any Zephyr features.

16.2.3 ZPL_BENCHMARK_RUN_TFLITE

TFLite-based Zephelin benchmark

Performs inference using a simple TFLite model. Uses ZPL scopes and model tracing.

16.2.4 ZPL_BENCHMARK_RUN_THREADS

Threads-based Zephelin benchmark

Features multiple threads performing a dummy operation on a single resource. Uses Zephyr's mutexes and semaphores.

16.3 Configs

16.3.1 instr_tracing.conf

```
CONFIG_TRACING=y
CONFIG_INSTRUMENTATION=y
CONFIG_TRACING_CTF=y
CONFIG_TRACING_SYNC=y
CONFIG_TRACING_MUTEX=y
CONFIG_INSTRUMENTATION_BACKEND_TRACING_CORE=y
CONFIG_THREAD_NAME=y
CONFIG_THREAD_MAX_NAME_LEN=20
CONFIG_INSTRUMENTATION_DYNAMIC_TRIGGER=n
CONFIG_MAIN_STACK_SIZE=8196
CONFIG_INSTRUMENTATION_TRIGGER_FUNCTION="benchmark_run"
CONFIG_INSTRUMENTATION_STOPPER_FUNCTION="benchmark_run"
```

16.3.2 prj.conf

```
CONFIG_TIMING_FUNCTIONS=y
```

16.3.3 tflite.conf

```
CONFIG_MAIN_STACK_SIZE=3072

CONFIG_TEST_RANDOM_GENERATOR=y
CONFIG_TENSORFLOW_LITE_MICRO=y
CONFIG_CPP=y
CONFIG_STD_CPP17=y
CONFIG_REQUIRES_FLOAT_PRINTF=y
```

16.3.4 tracing.conf

```
CONFIG_TRACING=y
CONFIG_TRACING_CTF=y
CONFIG_TRACING_SYNC=y
CONFIG_TRACING_MUTEX=y
CONFIG_TRACING_SEMAPHORE=y
CONFIG_TRACING_THREAD=y
CONFIG_TRACING_STACK=y
```

16.3.5 zpl.conf

```
CONFIG_TRACING=y
CONFIG_TRACING_CTF=y
CONFIG_TRACING_SYNC=y

CONFIG_ZPL=y
```

(continues on next page)

(continued from previous page)

```
CONFIG_ZPL_TRACE=y
CONFIG_ZPL_TRACE_FULL_MODE=y
CONFIG_ZPL_TRACE_FORMAT_CTF=y
CONFIG_ZPL_SCOPE_MARKING=y
```

16.4 Commits

Project	Commit
zephelin	073a6c3c0fa38366474e289279dfba6e567c6f1f
zephyr	3568e1b6d5cdd51a6b964a2a1d6d29200fea2056

INDEX

C

collect() (*in module handlers.trace_handler.TraceHandler*), 56

connect() (*in module handlers.trace_handler.TraceHandler*), 54

D

disconnect() (*in module handlers.trace_handler.TraceHandler*), 55

M

metadata() (*in module handlers.trace_handler.TraceHandler*), 55

R

reset() (*in module handlers.trace_handler.TraceHandler*), 55

S

stream_start() (*in module handlers.trace_handler.TraceHandler*), 55

stream_stop() (*in module handlers.trace_handler.TraceHandler*), 55